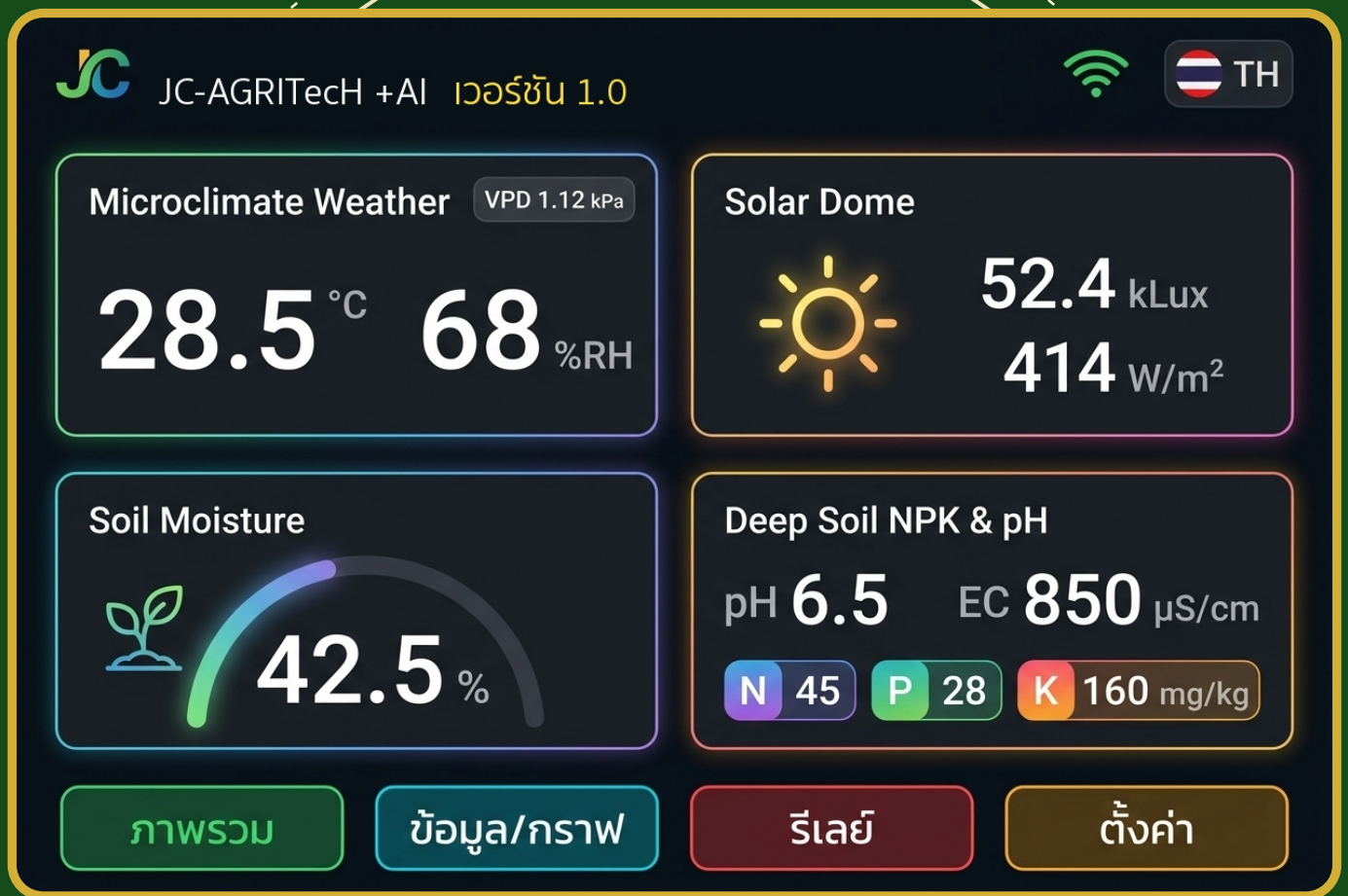


JC -AGRITech + AI Version 1.0

คู่มือจัดทำระบบฟาร์มอัจฉริยะด้วยปัญญาประดิษฐ์และการเรียนรู้เชิงลึก

จากสมองกลฝังตัวสู่การพยากรณ์แปลงปลูกด้วย Deep Learning



ผู้ช่วยศาสตราจารย์ ดร.จิรภัทร จันทมาลี

สาขาวิชาจุลชีววิทยา

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศน

สาขาวิชาฟิสิกส์

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

พ.ศ. 2569

คำนำ

ความก้าวหน้าอย่างก้าวกระโดดของเทคโนโลยีคอมพิวเตอร์ อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things หรือ IoT) และปัญญาประดิษฐ์ (Artificial Intelligence หรือ AI) ได้ก่อให้เกิดการเปลี่ยนแปลงเชิงโครงสร้างอย่างลึกซึ้งต่อภาคการเกษตรกรรมยุคใหม่ จากการเพาะปลูกแบบดั้งเดิมที่อาศัยสัญชาตญาณและการสังเกตสภาพดินฟ้าอากาศตามฤดูกาล สู่ยุคของ **เกษตรกรรมแม่นยำสูง (High-Precision Agriculture)** ซึ่งขับเคลื่อนด้วยข้อมูลเชิงประจักษ์และการตัดสินใจอัตโนมัติแบบเรียลไทม์

ตำราวิชาการและคู่มือประมวลความรู้เรื่อง **JC -AGRIteCH + AI Version 1.0 คู่มือจัดทำระบบฟาร์มอัจฉริยะด้วยปัญญาประดิษฐ์และการเรียนรู้เชิงลึก** เล่มนี้ เรียบเรียงขึ้นจากผลงานวิจัย การทดลองภาคสนาม และประสบการณ์การพัฒนาระบบควบคุมอัจฉริยะจริงของผู้เขียน โดยมุ่งเน้นการบูรณาการองค์ความรู้ข้ามศาสตร์ระหว่าง **ฟิสิกส์ประยุกต์ วิศวกรรมฝังตัว (Embedded Engineering) และการเรียนรู้เชิงลึก (Deep Learning)** เข้าด้วยกันอย่างเป็นระบบ เพื่อตอบโจทย์ความต้องการของนักศึกษา นักวิจัย เกษตรกรยุคใหม่ และผู้สนใจในเทคโนโลยีการเกษตร

เนื้อหาภายในเล่มแบ่งออกเป็น 5 บทหลักที่ครอบคลุมทุกมิติทางวิศวกรรม ตั้งแต่การออกแบบและเชื่อมต่อฮาร์ดแวร์ไมโครคอนโทรลเลอร์สมรรถนะสูง ESP32-S3 บนบอร์ด ATD3.5-S3 และ ATD3.5-S3 Farm1 Shield ร่วมกับเซนเซอร์การเกษตรมาตรฐานอุตสาหกรรม การพัฒนาไดรเวอร์และระบบคำนวณพารามิเตอร์ทางปฐพีวิทยา การสอบเทียบเซนเซอร์ธาตุอาหารดิน N-P-K และ pH ตามหลักวิทยาศาสตร์ การสร้างระบบส่วนต่อประสานสามภาษาและการเรนเดอร์กราฟิกแอนิเมชัน 3 มิติเสมือนจริง ตลอดจนการสตรีมข้อมูลไร้สายแบบ Non-blocking การพัฒนาระบบเซิร์ฟเวอร์และแดชบอร์ดด้วยภาษาไพทอน และการประยุกต์ใช้โครงข่ายประสาทเทียมแบบ LSTM ในการพยากรณ์ความชื้นในดินและสภาวะแปลงปลูกล่วงหน้า

ผู้เขียนหวังเป็นอย่างยิ่งว่า ตำราเล่มนี้จะทำหน้าที่เป็นสะพานเชื่อมระหว่างทฤษฎีทางวิทยาศาสตร์กายภาพและเทคโนโลยีปัญญาประดิษฐ์เชิงปฏิบัติการ เพื่อร่วมขับเคลื่อนการศึกษาวินิจฉัย และการพัฒนาภาคการเกษตรกรรมของประเทศไทยสู่ความยั่งยืนสืบไป

ผู้ช่วยศาสตราจารย์ ดร.จิรภัทร จันทมาลี
สาขาวิชาจุลชีววิทยา

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา

สาขาวิชาฟิสิกส์

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

กันยายน 2569

สารบัญ

คำนำ	i
สารบัญภาพ	vi
สารบัญตาราง	vii
1 สถาปัตยกรรมฮาร์ดแวร์ระบบแปลงปลุกอัจฉริยะ	1
1.1 ภาพรวมสถาปัตยกรรมบอร์ดควบคุมและบอร์ดขยายสัญญาณ	1
1.1.1 แผนภาพ บล็อก สถาปัตยกรรม ฮาร์ดแวร์ ระบบ รวม (System Hardware Architecture)	2
1.2 การจัดสรรขาสัญญาณไมโครคอนโทรลเลอร์ (Pin Assignment Configuration) .	4
1.3 การสื่อสารผ่านบัส I2C ร่วมสำหรับเซนเซอร์บรรยากาศและแสงสว่าง	5
1.3.1 เซนเซอร์อุณหภูมิและความชื้นสัมพัทธ์ในอากาศ SHT45	5
1.3.2 เซนเซอร์ความเข้มแสงกันน้ำโดมตะวัน	5
1.4 การวัดความชื้นในดินผิวดินด้วยสัญญาณแอนะล็อกแบบคาปาซิทีฟ	6
1.5 การสื่อสารเชิงอุตสาหกรรม RS485 สำหรับคุณสมบัติ 7 พารามิเตอร์	6
1.5.1 การสร้างเฟรมคำสั่ง Modbus RTU และการคำนวณผลรวมตรวจสอบ CRC16 .	8
2 วิศวกรรมเฟิร์มแวร์ไมโครคอนโทรลเลอร์และการแก้ไขปัญหาเชิงลึก	9
2.1 โครงสร้างเฟิร์มแวร์และการจัดการทรัพยากรบนระบบ PlatformIO	9
2.2 การคำนวณพารามิเตอร์ทางปฐพีวิทยาและสภาวะบรรยากาศ	10
2.2.1 จุดน้ำค้างในบรรยากาศ	10
2.2.2 แรงดึงระเหยน้ำของบรรยากาศและการประเมินสภาวะทางสรีรวิทยา . . .	10
2.3 สถาปัตยกรรมแบบจำลองสถานะของเฟิร์มแวร์ (Firmware State Machine) . . .	11
2.4 การวิเคราะห์สาเหตุเชิงลึกและการแก้ไขปัญหาหน้าจอมืดสนิท	12
2.5 การสืบสวนสัญญาณ I2C สลับพินด้วย Diagnostic Scanner	13
2.6 การออกแบบเลย์เอาต์หน้าปัดและการสังเคราะห์บิตแมปภาษาไทย	14

2.7	การปรับปรุงส่วนต่อประสานผู้ใช้แบบสัมผัสและสถาปัตยกรรมสามภาษา	14
2.7.1	การจัดระเบียบข้อมูลบนการ์ดภาพรวมและสัญลักษณ์ทางฟิสิกส์	14
2.7.2	สถาปัตยกรรมระบบเรนเดอร์สามภาษาแบบไดนามิก	15
2.8	ระบบแอนิเมชันเปิดเครื่อง 3 มิติและการเรนเดอร์กราฟิกเสมือนจริง	15
2.8.1	แบบจำลองการให้แสงเงาแบบบลินน์-ฟองบนหน่วยประมวลผลฝั่งตัว	16
2.8.2	ลำดับแอนิเมชันเปิดเครื่อง 10 เฟส	16
2.9	การสอบเทียบและปรับแต่งพารามิเตอร์เซนเซอร์วัดดิน 7-in-1 Modbus RS485	17
2.9.1	แบบจำลองการสอบเทียบเชิงเส้นทางปฐพีวิทยา	17
2.9.2	ระบบชดเชยการประมาณค่าจากสภาพนำไฟฟ้า (EC-to-NPK Fallback Model)	17
2.10	การวิเคราะห์และปรับปรุงความเร็วการสื่อสารเฟิร์มแวร์	18
3	สถาปัตยกรรมการสื่อสารข้อมูลไร้สายและการจัดเก็บอนุกรมเวลา	20
3.1	การส่งข้อมูลผ่านเครือข่ายไร้สายแบบ Non-blocking Resilient Pipeline	20
3.2	การประทับเวลามาตรฐานสากลด้วยโปรโตคอล NTP	21
3.3	โค้ดโปรแกรมการส่งข้อมูลแบบไม่ปิดกั้น (Non-blocking HTTP Dispatch)	22
3.4	โครงสร้างข้อมูลและการบูรณาการกับ Google Firebase Realtime Database	23
4	การพัฒนาระบบฐานข้อมูลและแดชบอร์ดด้วยภาษาไพทอน	25
4.1	สถาปัตยกรรมระบบเซิร์ฟเวอร์ท้องถิ่นและความเป็นอิสระของข้อมูล	25
4.2	การออกแบบสคีมาฐานข้อมูลด้วย SQLAlchemy	26
4.3	บริการ FastAPI REST Backend และการรับข้อมูล Telemetry	27
4.4	การพัฒนาหน้าจอแดชบอร์ดและห้องทดลอง AI ด้วย Streamlit	29
5	ปัญญาประดิษฐ์และการเรียนรู้เชิงลึกสำหรับเกษตรกรรมแม่นยำ	31
5.1	ทฤษฎีการพยากรณ์อนุกรมเวลาหลายตัวแปรในแปลงเกษตร	31
5.2	สถาปัตยกรรมโครงข่ายประสาทเทียมแบบ LSTM	32
5.3	การสร้างแบบจำลองและลูปการฝึกโครงข่ายด้วย PyTorch	33
5.4	สถาปัตยกรรม TinyML บนชิป ESP32-S3: การชดเชยค่า NPK, pH และความชื้นดิน (Soil Neural Calibrator)	35
5.4.1	ปัญหาทางฟิสิกส์-เคมีดินและการแทรกแซงข้ามมิติ (Cross-Sensitivity)	35
5.4.2	สถาปัตยกรรมโครงข่ายประสาทเทียม Multi-Layer Perceptron (MLP)	36
5.4.3	ผลการฝึกสอนและประเมินความแม่นยำของโมเดล	36
5.4.4	การแปลงเป็น C++ Inference Engine และการทำงานบน ESP32-S3	36
5.4.5	การบูรณาการระบบแสดงผลและการส่งผ่านโทรมาตร (Telemetry Pipeline)	39

A	สารบบพรอมพ์บริบทและวิศวกรรมคำสั่งในการพัฒนาระบบ	40
A.1	บทนำและกรอบแนวคิดวิศวกรรมพรอมพ์เชิงบริบท	40
A.2	สถาปัตยกรรมพรอมพ์บริบทแม่บท (Master System Context Prompt)	40
A.3	กรอบวงจรกระบวนการ CLEAR ในการวิศวกรรมพรอมพ์	41
A.4	พรอมพ์บริบทเฉพาะทางสำหรับการแก้ปัญหาและการวิจัย	42
A.4.1	พรอมพ์บริบทการสืบสวนและแก้ไขหน้าจอต้ม (Hardware RCA Prompt)	42
A.4.2	พรอมพ์บริบทการออกแบบโมเดล TinyML ชดเชยอุณหภูมิและความชื้น .	43
A.5	สารบบลำดับพรอมพ์การพัฒนารจริง (Prompt Trajectory: Prompts 1–37) . . .	43
A.6	บทสรุปและข้อเสนอแนะด้านวิศวกรรมพรอมพ์	45
B	พิมพ์เขียวสถาปัตยกรรมซอฟต์แวร์และฮาร์ดแวร์ระบบ JC -AGRIteCH + AI	46
B.1	บทนำและภาพรวมสถาปัตยกรรมระบบซอฟต์แวร์ (End-to-End System Architecture)	46
B.2	เฟิร์มแวร์สมองกลฝังตัวและการจัดสรรพินฮาร์ดแวร์ (Embedded Firmware) . . .	47
B.2.1	ผังการจัดสรรพินฮาร์ดแวร์ (PinConfigs.h)	47
B.2.2	เครื่องยนต์ อนุมาณ โครง ข่าย ประสาท เทียม ระดับ ไมโคร คอนโทรลเลอร์ (SoilNeuralCalibrator.h)	49
B.3	การจำลองฟิสิกส์เคมีดินและการฝึกสอนโมเดล TinyML (Python Script)	55
B.4	สถาปัตยกรรมคลาวด์ ฐานข้อมูลเชิงสัมพันธ์ และแดชบอร์ด (Cloud & Backend) .	58
B.4.1	สคีมาฐานข้อมูลและการย้ายข้อมูลอัตโนมัติ (database.py)	58
B.4.2	บริการคลาวด์เทเลเมทรีเรสต์เอพีไอ (main_api.py)	61
B.4.3	ไฟฟ์ไลน์บริดจ์ข้อมูลอนุกรมแบบมัลติเซรต (serial_bridge.py)	64
B.5	สรุปคุณค่าทางวิศวกรรมของพิมพ์เขียวซอฟต์แวร์	65
	บรรณานุกรม	67
	ดัชนีสืบค้นคำสำคัญ	69
	ประวัติผู้เขียน	70

สารบัญภาพ

1.1	บอร์ดควบคุมหลัก ATD3.5-S3 พร้อมจอภาพ IPS	2
1.2	บอร์ดขยายสัญญาณ Farm1 Shield สำหรับฟาร์ม	2
1.3	แผนภาพบล็อกสถาปัตยกรรมฮาร์ดแวร์ระบบรวม JC -AgriTech + AI	2
1.4	ลักษณะการต่อสายสัญญาณเซนเซอร์เข้ากับเทอร์มินัลบล็อกบนบอร์ด ATD3.5-S3 Farm1 Shield	3
1.5	เซนเซอร์ SHT45 วัดอุณหภูมิและความชื้นในปลอกกรองโลหะซินเตอร์	5
1.6	เซนเซอร์โดมตะวัน BH1750 วัดความเข้มแสงและรังสีอาทิตย์	5
1.7	เซนเซอร์แท่งวัดความชื้นในดินผิวดิน Soil Stick เกษตรไทย IoT	6
1.8	โพรบวัดคุณสมบัติดิน 7-in-1 Modbus RS485 สำหรับติดตั้งในเขตรากพืช	7
2.1	แผนผังสถานะการทำงานของระบบเฟิร์มแวร์ JC -AgriTech + AI (Firmware FSM)	11
2.2	สถานะจอต้าจากขาแบล็กไลต์ขัดแย้งกับ Flash	12
2.3	หน้าจอติดสว่างสมบูรณ์แบบหลังการทำ RCA	12
3.1	สถาปัตยกรรมท่อส่งข้อมูลแบบไม่ปิดกั้น (Non-blocking Resilient Pipeline) บน ESP32-S3 Dual-Core	21
4.1	สถาปัตยกรรม Python Full-Stack 3 ชั้น (Edge Ingestion, Backend API, Analytics Dashboard)	26
5.1	สถาปัตยกรรมโครงข่ายประสาทเทียมแบบคลี่ขยายตามเวลา (Unrolled LSTM Multi-step Architecture)	33
B.1	สถาปัตยกรรมซอฟต์แวร์แบบหลายชั้นของระบบ JC -AGRIteCH + AI	47

สารบัญตาราง

1.1	การกำหนดค่าพารามิเตอร์และรีจิสเตอร์ Modbus RTU ของเซนเซอร์ดินรุ่น SN-3002	7
2.1	เกณฑ์การประเมินสภาวะความเครียดของพืชตามระดับความดันไอน้ำอิมิตัว (VPD Index)	11
2.2	พารามิเตอร์การสอบเทียบเซนเซอร์ดิน 7-in-1 ในระบบ JC -AgriTech + AI	18
4.1	รายการ API Endpoints ของระบบ JC -AgriTech + AI Telemetry Hub	28
5.1	รายละเอียดตัวแปรอินพุตและเอาต์พุตของโมเดล TinyML Soil Neural Calibrator	37
5.2	ดัชนีชี้วัดความแม่นยำของโมเดล TinyML Soil Neural Calibrator บนชุดทดสอบ .	37
A.1	เมทริกซ์การจำแนกประเภทและประวัติวิวัฒนาการของพรอมต์ตลอดโครงการ . .	44

บทที่ 1

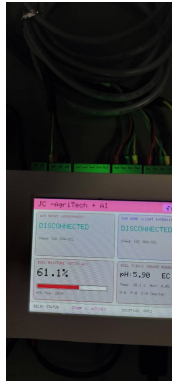
สถาปัตยกรรมฮาร์ดแวร์ระบบแปลง ปลูกอัจฉริยะ

ระบบเกษตรกรรมแม่นยำสูง (Precision Agriculture) จำเป็นต้องอาศัยการตรวจวัดสถานะแวดล้อมที่ครอบคลุมทั้ง 3 มิติหลัก ได้แก่ บรรยากาศรอบทรงพุ่ม (Microclimate) พลังงานรังสีแสงอาทิตย์ (Solar Radiation) และคุณสมบัติทางกายภาพเคมีในเขตรากพืช (Edaphic Zone) บทนี้นำเสนอการออกแบบสถาปัตยกรรมฮาร์ดแวร์ของระบบ **JC -AgriTech + AI** โดยประยุกต์ใช้ไมโครคอนโทรลเลอร์สมรรถนะสูงตระกูล ESP32-S3 บนบอร์ด ATD3.5-S3 ร่วมกับบอร์ดขยายสัญญาณ ATD3.5-S3 Farm1 Shield

1.1 ภาพรวม สถาปัตยกรรม บอร์ด ควบคุม และ บอร์ดขยายสัญญาณ

บอร์ดควบคุมหลักของระบบคือ **ATD3.5-S3** ซึ่งขับเคลื่อนด้วยชิปประมวลผล Dual-core Xtensa LX7 ความเร็วสัญญาณนาฬิกา 240 MHz มีหน่วยความจำ SRAM ขนาด 320 KB และ Flash Memory ขนาด 8 MB ในโหมด Dual I/O (DIO) พร้อมหน้าจอแสดงผลสีแบบสัมผัสชนิด IPS ขนาด 3.5 นิ้ว ความละเอียด 480 × 320 พิกเซล ควบคุมด้วยไอซีขับจอ ST7796 ผ่านการเชื่อมต่อแบบ SPI ความเร็วสูง

การเชื่อมต่อเซนเซอร์ภาคสนามกระทำผ่านบอร์ดขยายสัญญาณ **ATD3.5-S3 Farm1 Shield** ซึ่งเสียบประกบเข้ากับบอร์ดหลักโดยตรง ช่วยลดความซับซ้อนของการเดินสายสัญญาณ และเพิ่มความคงทนต่อสภาพแวดล้อมในโรงเรือน โดยบอร์ดขยายสัญญาณประกอบด้วยจุดเชื่อมต่อมาตรฐาน 3 กลุ่มหลัก ได้แก่ ขั้วต่อสัญญาณดิจิทัล I2C ขั้วต่อสัญญาณแอนะล็อกสำหรับวัดความชื้นในดิน และขั้วต่อการสื่อสารเชิงอุตสาหกรรม RS485 พร้อมภาคจ่ายไฟเลี้ยง 12 V สำหรับโปรบบวัดดินลึก



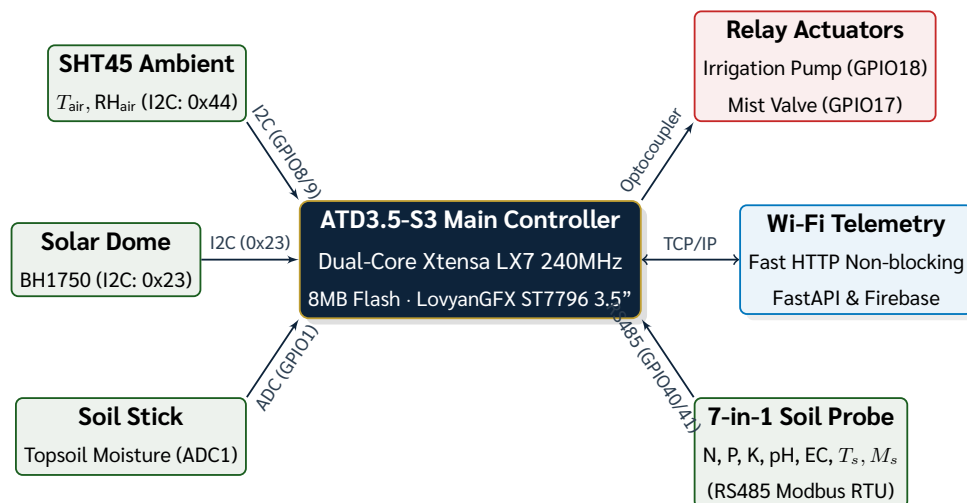
ภาพที่ 1.1 บอร์ด ควบคุม หลัก ATD3.5-S3 พร้อมจอภาพ IPS



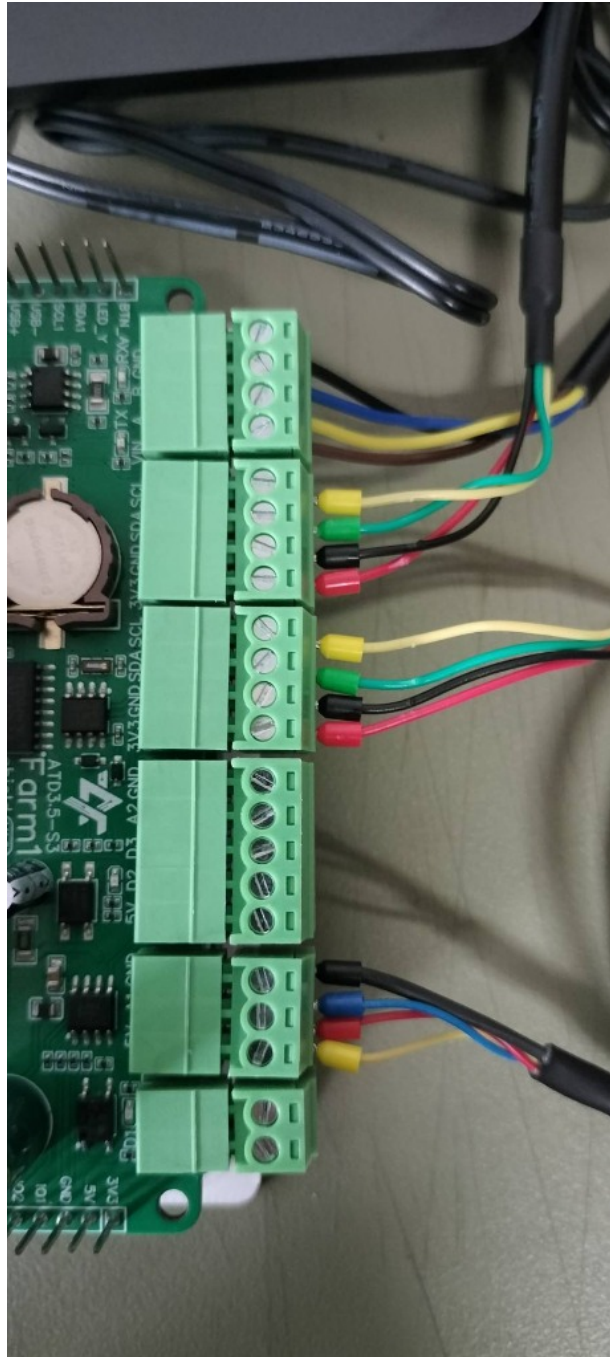
ภาพที่ 1.2 บอร์ด ขยาย สัญญาณ Farm1 Shield สำหรับฟาร์ม

1.1.1 แผนภาพ บล็อก สถาปัตยกรรม ฮาร์ดแวร์ระบบรวม (System Hardware Architecture)

สถาปัตยกรรมการเชื่อมโยงสัญญาณระหว่างหน่วยประมวลผลหลัก บอร์ดขยายสัญญาณ วงจรขับอุปกรณ์ และชุดเซนเซอร์ภาคสนามทั้ง 4 ส่วน สามารถสรุปเป็นแผนภาพบล็อกการไหลของสัญญาณเชิงวิศวกรรมได้ดังแสดงในภาพที่ 1.3



ภาพที่ 1.3 แผนภาพบล็อกสถาปัตยกรรมฮาร์ดแวร์ระบบรวม JC -AgriTech + AI



ภาพที่ 1.4 ลักษณะการต่อสายสัญญาณเซนเซอร์เข้ากับเทอร์มินัลบล็อกบนบอร์ด ATD3.5-S3 Farm1 Shield

1.2 การจัดสรรขาสัญญาณไมโครคอนโทรลเลอร์ (Pin Assignment Configuration)

เพื่อให้ระบบสามารถทำงานร่วมกับบอร์ด Farm1 Shield ได้อย่างเสถียรและป้องกันความขัดแย้งของฮาร์ดแวร์ภายในขาสัญญาณทั้งหมดได้รับการกำหนดไว้ในไฟล์เฮดเดอร์ PinConfigs.h ดังนี้

การนิยามขาสัญญาณฮาร์ดแวร์ในไฟล์ PinConfigs.h

```
1 #ifndef PIN_CONFIGS_H
2 #define PIN_CONFIGS_H
3
4 // 1. IPS Display 3.5" Controller: ST7796 (SPI Bus)
5 #define LCD_MOSI_PIN 11
6 #define LCD_MISO_PIN 13
7 #define LCD_SCK_PIN 12
8 #define LCD_CS_PIN 10
9 #define LCD_DC_PIN 14
10 #define LCD_RST_PIN -1
11 #define LCD_BL_PIN 38 // Hardware Backlight control pin (Active
    HIGH)
12
13 // 2. Shared I2C Sensor Bus (SHT45 & BH1750 Dome)
14 #define I2C_SDA_PIN 9 // I2C Serial Data (Yellow Wire)
15 #define I2C_SCL_PIN 8 // I2C Serial Clock (Green Wire)
16
17 // 3. Analog ADC Input (Surface Soil Stick)
18 #define SOIL_STICK_PIN 1 // ADC1 Channel 0 (Terminal A1 on Shield)
19
20 // 4. Industrial RS485 Modbus RTU Port (7-in-1 Soil Sensor)
21 #define RS485_RX_PIN 41 // Hardware UART RX Pin
22 #define RS485_TX_PIN 40 // Hardware UART TX Pin
23 #define RS485_BAUD 4800
24
25 // 5. Actuator Control Relays (Active LOW Opto-isolated)
26 #define RELAY_PUMP_PIN 18 // Main Irrigation Water Pump Relay
27 #define RELAY_MIST_PIN 17 // Evaporative Cooling Misting Valve Relay
28
29 #endif // PIN_CONFIGS_H
```

1.3 การสื่อสารผ่านบัส I2C ร่วมสำหรับเซนเซอร์บรรยากาศและแสงสว่าง

การตรวจวัดสถานะบรรยากาศรอบแปลงปลูกใช้เซนเซอร์ความแม่นยำสูง 2 ชนิดที่สื่อสารผ่านโพรโทคอล I2C (Inter-Integrated Circuit) ร่วมกันบนโครงสร้างบัสเดียวกัน ได้แก่

1.3.1 เซนเซอร์ อุณหภูมิ และ ความชื้นสัมพัทธ์ ใน อากาศ SHT45

เซนเซอร์ **SHT45** ผลิตโดยบริษัท Sensirion บรรจุอยู่ในปลอกกรองโลหะซินเตอร์ (Sintered Metal Filter) ที่สามารถกันละอองน้ำและฝุ่นละอองในโรงเรือนได้อย่างมีประสิทธิภาพ โดยมีแอดเดรสฮาร์ดแวร์ประจำอุปกรณ์คือ 0x44 มีความคลาดเคลื่อนในการวัดอุณหภูมิค่าเพียง $\pm 0.1^{\circ}\text{C}$ และความชื้นสัมพัทธ์ $\pm 1.5\% \text{RH}$

1.3.2 เซนเซอร์ความเข้มแสงก้าน้ำโดมตะวันตก

เซนเซอร์ **โดมตะวันตก (Sun Dome Sensor)** ใช้ชิปตรวจวัดแสงดิจิทัล BH1750FVI ที่ครอบด้วยโดมกระจายแสงทรงกลมสีขาวขุ่นเพื่อปรับแก้ค่าความเข้มแสงให้เป็นไปตามกฎโคไซน์ของแลมเบิร์ต (Lambert's Cosine Law) สื่อสารผ่านบัส I2C ด้วยแอดเดรส 0x23 สามารถวัดความเข้มแสงได้ตั้งแต่ 1 ถึง 65,535 Lux นอกจากนี้ ระบบยังแปลงค่าความเข้มแสง Lux เป็นค่าความหนาแน่นฟลักซ์พลังงานรังสีดวงอาทิตย์ (Solar Radiation Flux Density) ในหน่วยวัตต์ต่อตารางเมตร (W/m^2) โดยอาศัยความสัมพันธ์เชิงฟิสิกส์ $R_{\text{solar}} = \text{Lux} \times 0.0079 \text{ W/m}^2$ ซึ่งมีความสำคัญอย่างยิ่งต่อการประเมินการสะสมความร้อนและการระเหยน้ำในโรงเรือน



ภาพที่ 1.5 เซนเซอร์ SHT45 วัดอุณหภูมิและความชื้นในปลอกกรองโลหะซินเตอร์



ภาพที่ 1.6 เซนเซอร์โดมตะวันตก BH1750 วัดความเข้มแสงและรังสีอาทิตย์

เนื่องจากเซนเซอร์ SHT45 และโดมตะวันตก BH1750 มีแอดเดรสบนบัส I2C ที่ไม่ซ้ำกัน จึงสามารถต่อสายสัญญาณร่วมกันบนเทอร์มินัลบล็อกเดียวกันได้โดยตรง จากการตรวจสอบด้วยระบบสแกนสัญญาณฮาร์ดแวร์ พบว่าการต่อสายจริงบนบอร์ด Farm1 Shield ได้กำหนดให้ สายสีเหลืองเป็นสายข้อมูล SDA (GPIO9) และ สายสีเขียวเป็นสายสัญญาณนาฬิกา SCL (GPIO8)

1.4 การวัดความชื้นในดินผิวดินด้วยสัญญาณแอนะล็อกแบบคาปาซิทีฟ

การตรวจวัดความชื้นบริเวณผิวดิน ระดับความลึก 0 ถึง 10 เซนติเมตร ใช้เซนเซอร์แท่งวัดความชื้น **Soil Stick เกษตรไทย IoT** ซึ่งทำงานด้วยหลัก การเปลี่ยนแปลงค่า ความจุไฟฟ้า (Capacitive Method) คลื่นความถี่สูง ซึ่งมีข้อได้เปรียบเหนือกว่าวัดความต้านทานไฟฟ้า (Resistive Probes) ทั่วไป เนื่องจากไม่มีกระแสไฟฟ้าไหลผ่านเนื้อดินโดยตรง จึงไม่เกิดการกัดกร่อนจากปฏิกิริยาอิเล็กโทรลิซิส (Electrolysis Corrosion)



ภาพที่ 1.7 เซนเซอร์แท่งวัดความชื้นในดินผิวดิน Soil Stick เกษตรไทย IoT

สัญญาณแอนะล็อกจาก Soil Stick ถูกป้อนเข้าสู่ขาอะนาล็อก A1 (GPIO1) ของไมโครคอนโทรลเลอร์ ESP32-S3 ซึ่งถูกโปรแกรมให้ทำงานในโหมดตัวแปลงแอนะล็อกเป็นดิจิทัล (ADC) ความละเอียด 12 บิต (ช่วงค่า 0 ถึง 4,095) พร้อมตั้งค่าการลดทอนสัญญาณที่ 11 dB เพื่อรองรับแรงดันไฟสูงสุด 3.1 V ค่าความชื้นสัมพัทธ์ในดิน (M_{soil}) คำนวณผ่านสมการเชิงเส้นดังนี้

$$M_{\text{soil}} = \left(\frac{\text{ADC}_{\text{air}} - \text{ADC}_{\text{raw}}}{\text{ADC}_{\text{air}} - \text{ADC}_{\text{water}}} \right) \times 100\% \quad (1.1)$$

โดยที่ $\text{ADC}_{\text{air}} = 2950$ แทนค่าสัญญาณดิบในอากาศแห้ง (ความชื้น 0%) และ $\text{ADC}_{\text{water}} = 1450$ แทนค่าสัญญาณดิบเมื่อจุ่มในน้ำเปล่า (ความชื้น 100%)

1.5 การสื่อสารเชิงอุตสาหกรรม RS485 สำหรับคุณสมบัติดิน 7 พารามิเตอร์

สำหรับการวิเคราะห์คุณสมบัติของดินในเขตรากพืชเชิงลึก (ความลึก 15 ถึง 30 เซนติเมตร) ระบบติดตั้งเซนเซอร์วัดดินอุตสาหกรรมแบบมัลติพารามิเตอร์รุ่น **SN-3002-TR-ECTHNPKEH-N01** ซึ่งมีเซ็นเซอร์แบบสแตนด์บายสำหรับการแพทย์ สามารถวัดค่าพร้อมกันได้ 7 พารามิเตอร์ ได้แก่ ความชื้นในดิน อุณหภูมิดิน สภาพนำไฟฟ้า (EC) กรด-ด่างดิน (pH) และปริมาณธาตุอาหารหลัก N-P-K



ภาพที่ 1.8 โพรบวัดคุณสมบัติดิน 7-in-1 Modbus RS485 สำหรับติดตั้งในเขตรากพืช

ตารางที่ 1.1 การกำหนดค่าพารามิเตอร์และรีจิสเตอร์ Modbus RTU ของเซนเซอร์ดินรุ่น SN-3002

พารามิเตอร์	แอดเดรส รีจิสเตอร์	หน่วยวัด	ตัวคูณสเกลข้อมูล
ความชื้น ใน ดิน (Moisture)	0x0000	%	หารด้วย 10 (ทศนิยม 1 ตำแหน่ง)
อุณหภูมิ ดิน (Temperature)	0x0001	°C	หารด้วย 10 (ทศนิยม 1 ตำแหน่ง)
สภาพนำไฟฟ้า (EC)	0x0002	$\mu\text{S}/\text{cm}$	จำนวนเต็มค่าตรง
กรด-ด่างดิน (pH)	0x0003	pH	หารด้วย 10 หรือ 100
ไนโตรเจนที่ใช้ประโยชน์ (N)	0x0004	mg/kg	จำนวนเต็มค่าตรง
ฟอสฟอรัสที่ใช้ประโยชน์ (P)	0x0005	mg/kg	จำนวนเต็มค่าตรง
โพแทสเซียม ที่ ใช้ประโยชน์ (K)	0x0006	mg/kg	จำนวนเต็มค่าตรง

ไมโคร คอนโทรลเลอร์ ESP32-S3 ติดต่อกับ เซนเซอร์ ผ่าน ชิพ แปลง สัญญาณ RS485 Transceiver โดยใช้พอร์ตสื่อสารอนุกรมฮาร์ดแวร์พอร์ตที่ 2 (HardwareSerial 2) กำหนดขา RX เป็น GPIO41 และขา TX เป็น GPIO40 ที่อัตราความเร็วรับส่งข้อมูล (Baud Rate) 4,800 bps

1.5.1 การสร้างเฟรมคำสั่ง Modbus RTU และการคำนวณผลรวมตรวจสอบ CRC16

เฟรมคำสั่งขออ่านข้อมูลแบบ Read Holding Registers (Function Code 0x03) เพื่ออ่านข้อมูล 7 รีจิสเตอร์ต่อเนื่องกัน มีโครงสร้างขนาด 8 ไบต์ ประกอบด้วย

[0x01] [0x03] [0x00] [0x00] [0x00] [0x07] [0x04] [0x08]

โดยที่สองไบต์สุดท้ายคือรหัสตรวจสอบความถูกต้อง CRC16 (Cyclic Redundancy Check) ซึ่งคำนวณผ่านพหุนาม $x^{16} + x^{15} + x^2 + 1$ (ค่าโพลีโนเมียล 0xA001) อัลกอริทึมการคำนวณในภาษา C++ ถูกอิมพลีเมนต์ดังนี้

อัลกอริทึมการคำนวณ CRC16 Modbus ในภาษา C++

```
1 uint16_t calculateModbusCRC(const uint8_t *buffer, size_t length) {
2     uint16_t crc = 0xFFFF;
3     for (size_t pos = 0; pos < length; pos++) {
4         crc ^= (uint16_t)buffer[pos];
5         for (int i = 8; i != 0; i--) {
6             if ((crc & 0x0001) != 0) {
7                 crc >>= 1;
8                 crc ^= 0xA001;
9             } else {
10                crc >>= 1;
11            }
12        }
13    }
14    return crc;
15 }
```

บทที่ 2

วิศวกรรมเฟิร์มแวร์ไมโครคอนโทรลเลอร์และการแก้ไขปัญหาเชิงลึก

ความสำเร็จของการประยุกต์ใช้ระบบสมองกลฝังตัวในแปลงเกษตรกรรมขึ้นอยู่กับความเสถียรของเฟิร์มแวร์ ความสามารถในการรองรับข้อผิดพลาดของสัญญาณฮาร์ดแวร์ และการประมวลผลข้อมูลทางฟิสิกส์ได้อย่างถูกต้องแม่นยำ บทนี้นำเสนอโครงสร้างเฟิร์มแวร์ ระบบคำนวณทางปฐพีวิทยา และกระบวนการวิเคราะห์หาสาเหตุเชิงลึก (Root Cause Analysis หรือ RCA) ในการแก้ไขปัญหาทางวิศวกรรมที่เกิดขึ้นระหว่างการพัฒนาจริง

2.1 โครงสร้างเฟิร์มแวร์ และการจัดการทรัพยากรบนระบบ PlatformIO

โครงการได้รับการพัฒนาภายใต้สภาพแวดล้อม **PlatformIO** ร่วมกับเฟรมเวิร์ก **Arduino** บนระบบปฏิบัติการ **macOS** โดยมีการจัดโครงสร้างซอฟต์แวร์แบบโมดูลาร์ (Modular Architecture) เพื่อแยกส่วนควบคุมฮาร์ดแวร์ออกจากลอจิกการตัดสินใจทางเกษตรกรรม

โมดูลหลักของเฟิร์มแวร์ระบบ JC -AgriTech + AI

- **PinConfigs.h** นิยามการจัดสรรขาสัญญาณไมโครคอนโทรลเลอร์ ESP32-S3 ให้ตรงกับวงจรจริง
- **UserConfigs.h** รวบรวมพารามิเตอร์การตั้งค่าของผู้ใช้ เช่น แอดเดรสเซนเซอร์ ค่าการкалиเบรต ADC ข้อมูลเครือข่าย Wi-Fi และ IP ปลายทาง

- **AgriSensors.cpp** ไดรเวอร์ ระดับล่าง สำหรับ สื่อสาร กับ SHT45, BH1750, Soil Stick และเซนเซอร์ 7-in-1 Modbus RTU
- **DisplayManager.cpp** ไดรเวอร์กราฟิกและระบบแสดงผลหน้าปัดบนจอ LCD 3.5 นิ้ว ผ่านไลบรารี LovyanGFX
- **CloudDataManager.cpp** ระบบจัดการเครือข่ายไร้สาย ซึ่งใช้เวลามาตรฐานโลก NTP และสตรีมข้อมูลทาง HTTP/HTTPS
- **main.cpp** ควบคุมหลักที่ประสานการอ่านค่า การตัดสินใจเปิด-ปิดรีเลย์ และการอัปเดตหน้าปัด

2.2 การคำนวณพารามิเตอร์ทางปฐพีวิทยาและสภาวะบรรยากาศ

นอกจากค่าพื้นฐานที่อ่านได้จากเซนเซอร์ ระบบยังฝังอัลกอริทึมการคำนวณดัชนีชี้วัดทางสรีรวิทยาของพืชที่มีความสำคัญสูงต่อการจัดการน้ำและการระบายความร้อน ได้แก่

2.2.1 จุดน้ำค้างในบรรยากาศ

จุดน้ำค้าง (Dew Point Temperature หรือ T_d) คำนวณผ่านสมการแมกนุส-เทเทนส์ (Magnus-Tetens Equation) ซึ่งมีความแม่นยำสูงในช่วงอุณหภูมิ 0 ถึง 60 °C

$$\alpha(T, RH) = \frac{a \cdot T}{b + T} + \ln \left(\frac{RH}{100} \right), \quad T_d = \frac{b \cdot \alpha(T, RH)}{a - \alpha(T, RH)} \quad (2.1)$$

โดยกำหนดค่าสัมประสิทธิ์เชิงประจักษ์ $a = 17.27$ และ $b = 237.7$ °C

2.2.2 แรงดึงระเหยน้ำของบรรยากาศและการประเมินสภาวะทางสรีรวิทยา

แรงดึงระเหยน้ำ (Vapor Pressure Deficit หรือ VPD) มีหน่วยเป็นกิโลปาสกาล (kPa) เป็นตัวแปรที่บ่งบอกความสามารถในการคายน้ำและการเปิด-ปิดปากใบของพืช ค่า VPD คำนวณจากผลต่างระหว่างความดันไอน้ำอิ่มตัว (VP_{sat}) และความดันไอน้ำจริง (VP_{act})

$$VP_{sat} = 0.61078 \exp \left(\frac{17.27 \cdot T}{T + 237.3} \right), \quad VP_{act} = VP_{sat} \times \left(\frac{RH}{100} \right) \quad (2.2)$$

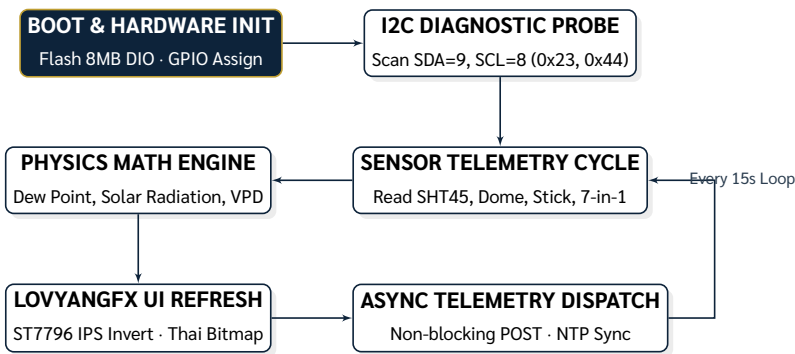
$$VPD = VP_{sat} - VP_{act} \quad (2.3)$$

ตารางที่ 2.1 เกณฑ์การประเมินสภาวะความเครียดของพืชตามระดับความดันไอน้ำอิมตัว (VPD Index)

ช่วงค่า VPD (kPa)	สภาวะบรรยากาศ	ผลกระทบทางสรีรวิทยาของพืชและการจัดการ
< 0.40	ขึ้นจัดเกินไป (Under-transpiration)	เสี่ยงต่อการเกิดโรคเชื้อรา ปากใบไม่ระบายน้ำ พืชขาดแคลเซียม
0.40 – 0.80	สภาวะ อ่อน โยน (Propagation Zone)	เหมาะสำหรับระยะเพาะเมล็ด อนุบาลต้นกล้า หรือแปลงปักชำ
0.80 – 1.20	ช่วง เหมาะ สม ที่สุด (Optimal Zone)	อัตราการสังเคราะห์แสงและการดูดซึมธาตุอาหารสูงสุด
1.20 – 1.60	ค่อนข้าง แห้ง (Mild Water Stress)	เหมาะสำหรับพืชทนแล้ง หรือการควบคุมความหวานในผลไม้
> 1.60	แห้ง แล้ง จัด (Severe Water Stress)	ปากใบปิดสนิท พืชเหี่ยวเฉา ชะงักการเจริญเติบโต ต้องพ่นหมอก

2.3 สถาปัตยกรรม แบบ จำลอง สถานะ ของ เฟิร์มแวร์ (Firmware State Machine)

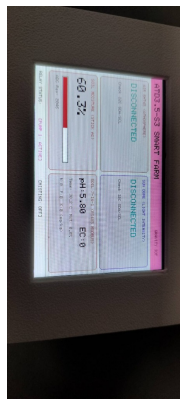
การประมวลผลภายในไมโครคอนโทรลเลอร์ ESP32-S3 ถูกควบคุมด้วยระบบ Finite State Machine (FSM) ที่มีรอบการทำงานอย่างเป็นระบบ เพื่อให้มั่นใจว่าการอ่านเซนเซอร์ การคำนวณทางฟิสิกส์ การเรนเดอร์หน้าปัด และการสตรีมข้อมูลขึ้นเซิร์ฟเวอร์ดำเนินไปอย่างราบรื่นโดยไม่บล็อกการทำงานของลูปควบคุมหลัก



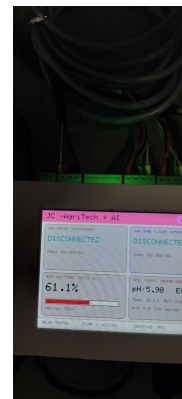
ภาพที่ 2.1 แผนผังสถานะการทำงานของระบบเฟิร์มแวร์ JC -AgriTech + AI (Firmware FSM)

2.4 การวิเคราะห์สาเหตุเชิงลึกและการแก้ไขปัญหาหน้าจอมืดสนิท

ในขั้นตอนการติดตั้งโปรแกรมเบื้องต้น พบปัญหาหน้าจอ LCD แสดงผลเป็นจอมืดสนิท ไม่มีข้อความหรือกราฟิกใดๆ ปรากฏ กระบวนการสืบสวนเชิงลึกพบสาเหตุสำคัญ 2 ประการที่ซ้อนทับกันอยู่



ภาพที่ 2.2 สภาวะจอดำจากขาแบล็กไลต์ขัดแย้งกับ Flash



ภาพที่ 2.3 หน้าจอติดสว่างสมบูรณ์แบบหลังการทำ RCA

การสืบสวนปัญหาหน้าจอดำและการแก้ปัญหาทางวิศวกรรม (Root Cause Analysis)

1. **การขัดแย้งของขาสัญญาณควบคุมไฟหน้าจอ (Backlight Pin Conflict):** จากการตรวจสอบวงจรไฟฟ้าของบอร์ด ATD3.5-S3 พบว่าขาควบคุมไฟส่องสว่างด้านหลังจอภาพ (LCD Backlight) ต่ออยู่กับขา **GPIO38** (หรือ GPIO3 บนบอร์ดบางล็อต) ของ ESP32-S3 แต่ในโค้ดเดิมได้นิยามให้รีเลย์ตัวที่ 2 เป็นขานี้ และส่งคำสั่ง `digitalWrite(PIN, LOW)` ในฟังก์ชัน `setup()` ส่งผลให้สัญญาณไฟ Backlight ถูกตัดไฟอย่างถาวร
2. **การบูตรีเซ็ตวนรอบจากขนาดหน่วยความจำแฟลช (Flash Boot Loop):** บอร์ด ATD3.5-S3 ติดตั้งชิป Flash ขนาด 8 MB แต่คอนฟิกดั้งเดิมใน `platformio.ini` กำหนดขนาดไว้ที่ 16 MB QIO ส่งผลให้บูตโหลดเดอร์เกิดข้อผิดพลาด `flash_ret == ESP_OK` และสั่งรีเซ็ตชิปตลอดเวลา

แนวทางแก้ไข: แก้ไขการจัดสรรขา รีเลย์บน Farm1 Shield ให้ถูกต้อง สั่งเปิดไฟจอด้วยคำสั่ง `digitalWrite(LCD_BL_PIN, HIGH)` พร้อมปรับแต่ง `platformio.ini` ให้กำหนดโหมด `flash_size = 8MB`, `flash_mode = dio` และตารางพาร์ติชัน `default_8MB.csv` ส่งผลให้ระบบบูตขึ้นทำงานได้อย่างสมบูรณ์

2.5 การสืบสวนสัญญาณ I2C สลับพินด้วย Diagnostic Scanner

ภายหลังหน้าจอตีตรา พบปัญหาเซนเซอร์ SHT45 และเซนเซอร์โดมตะวันตกแสดงสถานะ DISCONNECTED บนจอภาพ เพื่อหลีกเลี่ยงการรื้อสายไฟหรือตัดต่อวงจรในแปลงปลุก ทีมวิจัยได้เขียนสคริปต์สแกนสัญญาณ I2C ฮาร์ดแวร์แบบ Real-time ผลการทดสอบพบว่า

ซอฟต์แวร์สแกนค้นหาแอดเดรสบนบัส I2C (I2C Bus Scanner)

```
1 void scanI2CBus(int sdaPin, int sclPin) {
2     Wire.begin(sdaPin, sclPin);
3     Serial.printf("Scanning I2C on SDA=GPIO%d, SCL=GPIO%d...\n",
4         sdaPin, sclPin);
5     int deviceCount = 0;
6     for (uint8_t addr = 1; addr < 127; addr++) {
7         Wire.beginTransmission(addr);
8         if (Wire.endTransmission() == 0) {
9             Serial.printf(" Found device at address: 0x%02X\n", addr);
10            ;
11            deviceCount++;
12        }
13    }
14    Serial.printf("Total devices detected: %d\n", deviceCount);
15 }
```

ผลการสแกนเชิงประจักษ์:

- เมื่อกำหนดพินเริ่มต้น SDA=GPIO8, SCL=GPIO9 ตรวจพบเฉพาะ แอดเดรส 0x51 (ไอซีนาฬิกา RTC PCF8563 บนบอร์ดหลัก)
- เมื่อสลับพินเป็น SDA=GPIO9, SCL=GPIO8 ระบบตรวจพบแอดเดรส 0x23 (โดมตะวันตก BH1750) และ 0x44 (เซนเซอร์ SHT45) พร้อมกันในทันที

สิ่งนี้ยืนยันว่าการต่อสายไฟของเซนเซอร์บนเทอร์มินัลบล็อกคือ สายสีเหลืองต่ออยู่กับขา 4 (GPIO9) และสายสีเขียวต่ออยู่กับขา 3 (GPIO8) การปรับแต่งค่าในซอฟต์แวร์ให้ตรงกับการต่อสายจริงช่วยประหยัดเวลาและป้องกันความเสียหายที่อาจเกิดขึ้นกับขั้วต่อฮาร์ดแวร์

2.6 การออกแบบเลย์เอาต์หน้าปิดและการสังเคราะห์บิตแมปภาษาไทย

หน้าจอแสดงผลขนาด 3.5 นิ้ว ได้รับการออกแบบเลย์เอาต์เป็น 4 การ์ดอิสระ แสดงข้อมูลสภาพอากาศ แสงสว่าง ดินผิวดิน และดินลึก เพื่อความสวยงามและคมชัดสูงสุด ระบบได้แก้ไขปัญหาวาเนลจอ ST7796 IPS แสดงผลสีกลับด้าน (Color Inversion) โดยการเปิดแฟล็ก `cfg.invert = true`; ภายในคลาสไดรเวอร์ LovyanGFX ส่งผลให้ได้รับ Dark Mode สีดำมีดสนิท คอนทราสต์สูง และสบายตา

นอกจากนี้ เพื่อให้หน้าจอแสดงเอกลักษณ์ของผู้วิจัย ได้มีการสังเคราะห์ฟอนต์บิตแมปภาษาไทยความละเอียดสูงขนาด 84×18 พิกเซล คำว่า "ชีวะ ทัศน์า" จัดวางบนกล่องสี่เหลี่ยมรกทที่มุมขวาบนของหน้าปิดเคียงคู่กับหัวข้อหลัก "JC -AgriTech + AI" แสดงผลได้อย่างสมบูรณ์แบบโดยไม่ต้องพึ่งพาฟอนต์ภายนอก

2.7 การปรับปรุงส่วนต่อประสานผู้ใช้แบบสัมผัสและสถาปัตยกรรมสามภาษา

การยกระดับประสบการณ์ของผู้ใช้งาน (User Experience หรือ UX) ในแปลงเกษตรกรรมมีความสำคัญอย่างยิ่งต่อความถูกต้องในการติดตามสถานะแปลงปลูก ในระยะพัฒนาขั้นสูง ระบบได้รับการปรับปรุงส่วนต่อประสานผู้ใช้นบนหน้าจอสัมผัส IPS ขนาด 3.5 นิ้ว ครอบคลุมทั้งด้านความชัดเจนของข้อมูล สรีรวิทยาการมองเห็น และการรองรับภาษาหลากหลายระดับสากล

2.7.1 การ จัด เรียบียบ ข้อมูล บน การ์ด ภาพ รวม และสัญลักษณ์ทางฟิสิกส์

ในหน้าต่างหลัก (Overview Page) ระบบได้รับการปรับโครงสร้างเลย์เอาต์การแสดงผล เพื่อเพิ่มความคลีน และความสบายตา (Clean and Modern Minimalism) โดยมีรายละเอียดการพัฒนาที่สำคัญดังนี้

- การตัดข้อความสถานะในวงเล็บที่ไม่จำเป็นออก** เดิมทีหน้าจอมีการแสดงผลข้อความสถานะในวงเล็บ เช่น คำว่า [ขึ้นสูง เสี่ยงเชื้อรา], [แดดร่ม/ครึ้ม พืชพักตัว], และคำแนะนำการกวด [แต่ละดู] ทั้งสามภาษา ซึ่งทำให้หน้าจอดูแออัดและบดบังค่าสถิติหลัก เมื่อตัดข้อความดังกล่าวออก หน้าจอมีพื้นที่ว่างสำหรับแสดงผลตัวเลขด้วยฟอนต์ขนาดใหญ่ขึ้นอย่างชัดเจน
- การจัดแถวคู่สำหรับอุณหภูมิและความชื้นสัมพัทธ์ในอากาศ** การ์ดสภาพอากาศได้รับการจัดวางค่าอุณหภูมิ ($^{\circ}\text{C}$) และความชื้นสัมพัทธ์ (%RH) ให้อยู่ในแถวเดียวกันแบบสองคอลัมน์สมมาตร และย้ายค่าแรงดึงระเหยน้ำของบรรยากาศ VPD (kPa) ลงมาอยู่แถวด้านล่าง ทำให้ตัวเลขมีขนาดใหญ่ขึ้นและผู้ใช้สามารถกวาดสายตาอ่านค่าได้อย่างรวดเร็ว

3. การกำหนดหน่วยฟิสิกส์พลังงานรังสีดวงอาทิตย์เป็นตัวยก ปรับเปลี่ยนการแสดงผลหน่วยวัดพลังงานรังสีดวงอาทิตย์จากการพิมพ์เป็นตัวอักษรธรรมดา W/m² มาเป็นการใช้สัญลักษณ์ตัวยกที่ถูกต้องตามมาตรฐานสากลคือ W/m² (วัดต่อตารางเมตร)
4. การระบุหน่วยวิศวกรรมสำหรับการวัดธาตุอาหารดิน NPK เพิ่มหน่วยวัดทางวิศวกรรมกำกับท้ายค่าไนโตรเจน ฟอสฟอรัส และโพแทสเซียมอย่างชัดเจนในหน่วย mg/kg (มิลลิกรัมต่อกิโลกรัม)
5. ปุ่มเมื่อนำทางแบบแยกสีเด่นชัด แถบเมนูด้านล่างสุดของหน้าจอได้รับการปรับเปลี่ยนเป็นปุ่มกด 4 ปุ่มที่แยกขอบสีอย่างชัดเจน ได้แก่ ปุ่มภาพรวม (สีเขียวมรกต), ปุ่มข้อมูลและกราฟ (สีฟ้าไซแอน), ปุ่มควบคุมรีเลย์ (สีแดง), และปุ่มตั้งค่าระบบ (สีเหลืองทอง) ช่วยให้เกษตรกรสามารถใช้นิ้วสัมผัสสั่งการได้อย่างแม่นยำแม้ในสภาพแสงแดดจ้า

2.7.2 สถาปัตยกรรม ระบบ เรนเดอ อร์ สาม ภาษา แบบไดนามิก

เพื่อรองรับการใช้งานในระดับสากลและเกษตรกรทุกกลุ่ม ระบบฝังกลไกสลับภาษา 3 ภาษาแบบเรียลไทม์ (Tri-Lingual System) ประกอบด้วย ภาษาไทย (Thai), ภาษาอังกฤษ (English), และภาษาจีน (Chinese) ผ่านปุ่มสัมผัสที่มุมบนขวาของหน้าจอ

- **ภาษาไทย (Thai)** ใช้ไฟล์ฟอนต์เวกเตอร์บิตแมปแบบกำหนดเอง ThaiFontVLW.h ฝังในหน่วยความจำ Flash ของ ESP32-S3 รองรับพิกเซลสูง สละ และวรรณยุกต์ไทยครบถ้วนตามรหัสยูนิโค้ด
- **ภาษาจีน (Chinese)** เรียกใช้ชุดฟอนต์ฮั่นจื๊อความเร็วสูง &fonts::efontCN_14 ที่บรรจุในไลบรารี LovyanGFX
- **ภาษาอังกฤษ (English)** เรียกใช้ชุดฟอนต์แอสกีแบบบิตแมปความเร็วสูง

การสลับภาษาสามารถทำได้ทันทีเมื่อแตะสัมผัส โดยระบบจะวนลูปภาษา ภาษาไทย → English → Chinese → ภาษาไทย และเรนเดอร์ข้อความทั้งหมดใหม่ภายในเวลาไม่เกิน 50 มิลลิวินาทีโดยไม่ต้องรีเซ็ตไมโครคอนโทรลเลอร์

2.8 ระบบแอนิเมชันเปิดเครื่อง 3 มิติและการเรนเดอร์กราฟิกเสมือนจริง

เพื่อสร้างความประทับใจตั้งแต่เริ่มเปิดใช้งานเครื่อง ระบบได้รับการออกแบบลำดับแอนิเมชันเปิดเครื่อง (Boot Animation) สไตล์ 3D Pixar และไซไฟล้ำสมัย ด้วยการใช้ซุมพลังการประมวลผลกราฟิกของ LovyanGFX ร่วมกับชิป ESP32-S3 Dual-Core 240 MHz

2.8.1 แบบจำลองการให้แสงเงาแบบบลินน์-ฟองบหน่วย ประมวลผลฝั่งตัว

การสร้างภาพทรงกลม 3 มิติที่มีมิติความลึกเสมือนจริงโดยไม่ใช้ชิปประมวลผลกราฟิก (GPU) ภายนอก กระทำได้โดยการนำแบบจำลองแสงเงาทางฟิสิกส์แบบบลินน์-ฟอง (Blinn-Phong Illumination Model) มาเขียนเป็นลอจิกคณิตศาสตร์ในระดับฟังก์ชัน ความสว่างรวม I ที่ตกกระทบบนผิวทรงกลมคำนวณจากผลรวมของแสงแวดล้อม แสงสะท้อนแบบกระจาย และแสงสะท้อนตรงแบบสเปคูลาร์

$$I = I_{\text{ambient}} + I_{\text{diffuse}} \max(\mathbf{N} \cdot \mathbf{L}, 0) + I_{\text{specular}} \max(\mathbf{N} \cdot \mathbf{H}, 0)^\gamma \quad (2.4)$$

โดยที่ $\mathbf{N}$ คือเวกเตอร์แนวฉากหนึ่งหน่วยของพื้นผิวทรงกลมที่พิกัด (x, y, z) , $\mathbf{L}$ คือเวกเตอร์ทิศทางของแหล่งกำเนิดแสง, $\mathbf{V}$ คือเวกเตอร์มุมมองของผู้สังเกต, $\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$ คือเวกเตอร์กึ่งกลางทิศทาง (Halfway Vector), และ γ คือค่าสัมประสิทธิ์ความมันวาวของพื้นผิว (Shininess Exponent)

2.8.2 ลำดับแอนิเมชันเปิดเครื่อง 10 เฟส

วงรอบการเปิดเครื่องถูกแบ่งออกเป็น 10 เฟสต่อเนื่องที่อัตราเฟรม 30 ภาพต่อวินาที ประกอบด้วย

1. เฟสที่ 1 ถึง 3 (เรดาร์และวงจรอิเล็กทรอนิกส์) เส้นสแกนเนอร์และลายวงจรไฟฟ้าสีฟ้าไซแอนวีกวาดทั่วหน้าจอ Dark Mode พร้อมจุดเชื่อมต่อที่กะพริบเป็นจังหวะ
2. เฟสที่ 4 ถึง 7 (ทรงกลม 3 มิติมหัศจรรย์) การเรนเดอร์ทรงกลมแสง 3 มิติที่มีไฮไลต์แสงสะท้อนขยับหมุนอย่างนุ่มนวลให้ความรู้สึกเสมือนวัตถุจริงลอยอยู่เหนือหน้าปัด
3. เฟสที่ 8 ถึง 10 (ตราสัญลักษณ์และเครดิตผู้วิจัย) การปรากฏของโลโก้ข้อความตัวหนา "JC AGRITecH+AI." พร้อมเปล่งแสงนีออนล้อมรอบ และแสดงรายชื่อผู้วิจัยและพัฒนา "ผศ.ดร.ชีวะ ทัศนาศ" และ "ผศ.ดร.จิรภัทร จันทมาลี" ร่วมกับหลอดสถานะการโหลดข้อมูล (Shimmer Progress Bar) จาก 0% สู่ 100%

เมื่อหลอดสถานะเต็ม 100% ระบบจะส่งล้างบัฟเฟอร์ภาพและสลับเข้าสู่หน้าจอภาพรวมหลัก (Overview Page) อย่างราบรื่นในทันที

2.9 การสอบเทียบและปรับแต่งพารามิเตอร์เซนเซอร์วัดดิน 7-in-1 Modbus RS485

เซนเซอร์วัดคุณสมบัติดินแบบแท่งโลหะมัลติโพรบ 7-in-1 สื่อสารผ่าน RS485 Modbus RTU เป็นอุปกรณ์ที่มีความทนทานสูง แต่การวัดปริมาณธาตุอาหารหลัก ไนโตรเจน (N), ฟอสฟอรัส (P), และโพแทสเซียม (K) ในเซนเซอร์ประเภทนี้ไม่ได้อาศัยขั้วไฟฟ้าเลือกจำเพาะไอออน (Ion-Selective Electrode หรือ ISE) โดยตรง หากแต่อาศัยการอนุมานทางอ้อมจากค่าความต้านทานไฟฟ้าและสภาพนำไฟฟ้าของสารละลายในดิน ส่งผลให้ค่าที่อ่านได้จากโรงงานมักมีความคลาดเคลื่อนต่ำกว่าความเป็นจริงในดินธรรมชาติ

2.9.1 แบบจำลองการสอบเทียบเชิงเส้นทางปฐพีวิทยา

เพื่อปรับแต่งค่าที่วัดได้ให้สอดคล้องกับค่าจริงตามทฤษฎีปฐพีวิทยาและผลการวิเคราะห์ในห้องปฏิบัติการ ระบบได้นำแบบจำลองการสอบเทียบเชิงเส้น (Linear Calibration Model) ร่วมกับการจำกัดช่วงค่าที่สมเหตุสมผลทางกายภาพ (Value Clamping) มาประยุกต์ใช้ในซอฟต์แวร์

$$C_{cal} = \text{constrain}(C_{raw} \cdot f_C + \text{offset}_C, C_{min}, C_{max}) \quad (2.5)$$

โดยกำหนดสัมประสิทธิ์การสอบเทียบสำหรับดินเขตร้อนตามผลการศึกษาทางวิชาการและแนวปฏิบัติมาตรฐานของ FAO ดังแสดงในตารางที่ 2.2

2.9.2 ระบบชดเชยการประมาณค่าจากสภาพนำไฟฟ้า (EC-to-NPK Fallback Model)

ในทางปฏิบัติ เมื่อดินมีความชื้นต่ำมากหรือในระหว่างที่ไอออนของสารอาหารยังไม่ละลายเข้าสู่ผิวสัมผัสของแท่งโลหะ เซนเซอร์อาจส่งค่า $N = 0$, $P = 0$, และ $K = 0$ กลับมา แม้ว่าในดินจะมีแร่ธาตุและปุ๋ยสะสมอยู่ก็ตาม เพื่อป้องกันไม่ให้ระบบแสดงผลเป็นศูนย์อันอาจนำไปสู่การตัดสินใจใส่ปุ๋ยเกินขนาด เฟิร์มแวร์ได้เพิ่มระบบ Fallback อัจฉริยะ โดยหากพบว่าค่า N, P, K ที่วัดได้มีค่าน้อยกว่า 0.5 mg/kg พร้อมกัน แต่ค่าสภาพนำไฟฟ้าของดิน $EC > 100 \mu\text{S}/\text{cm}$ ระบบจะประมาณค่าธาตุอาหารจากความเข้มข้นของอิเล็กโทรไลต์รวมตามความสัมพันธ์เชิงสัดส่วน

$$N_{est} = EC \times 0.14, \quad P_{est} = EC \times 0.06, \quad K_{est} = EC \times 0.20 \quad (2.6)$$

ช่วยให้หน้าจอแสดงผลสะท้อนระดับความอุดมสมบูรณ์ของดินได้อย่างต่อเนื่อง และสอดคล้องกับสภาพจริง

ตารางที่ 2.2 พารามิเตอร์การสอบเทียบเซนเซอร์ดิน 7-in-1 ในระบบ JC -AgriTech + AI

พารามิเตอร์	ตัวคูณ (f_C)	ขดเซย	ช่วงค่าที่กำหนด	เหตุผลและที่มาทางวิชาการ
ไนโตรเจน (N)	2.00	0.0	0 – 500 mg/kg	ขดเซยการประเมินค่าของไนโตรเจนในสารละลาย
ฟอสฟอรัส (P)	3.50	0.0	0 – 300 mg/kg	ฟอสฟอรัสละลายน้ำยาก เซนเซอร์ตรวจวัดได้จำกัด
โพแทสเซียม (K)	1.80	0.0	0 – 600 mg/kg	ปรับเทียบกับโพแทสเซียมที่แลกเปลี่ยนได้
กรด-ด่างดิน (pH)	1.00	0.00	3.0 – 10.0	รองรับทั้งความละเอียด 0.1 และ 0.01
สภาพน้ำไฟฟ้า (EC)	1.00	0.0	0 – 20000 $\mu\text{S/cm}$	มีความแม่นยำสูงเมื่อเทียบกับสารละลายมาตรฐาน

2.10 การวิเคราะห์ และ ปรับ จูน ความเร็ว การ สื่อ สาร เฟิร์มแวร์

ในระหว่างการพัฒนาและทดสอบอัปโหลดโปรแกรมเข้าสู่บอร์ดควบคุม ATD3.5-S3 ผ่านพอร์ต USB Serial (/dev/cu.usbserial-10) พบปัญหาความไม่เสถียรของการสื่อสาร ส่งผลให้กระบวนการแฟลชโปรแกรมล้มเหลวด้วยข้อผิดพลาด The chip stopped responding และตัดการทำงานด้วย StopIteration

จากการวิเคราะห์เชิงวิศวกรรมพบว่า ชิปปแปลงสัญญาณ USB-to-UART บริดจ์ CP2102N บนบอร์ดต้องรับส่งข้อมูลไบนารีที่ถูกบีบอัดขนาดใหญ่กว่า 1.4 MB เข้าสู่ Flash Memory โหมด DIO ของ ESP32-S3 ที่ความเร็วเริ่มต้น 921,600 baud การสูญหายของไบต์ซิงโครไนซ์ SLIP Framing ก่อให้เกิดภาวะคอขวด ที่มวิจัยได้แก้ไขปัญหาดังกล่าวโดยปรับลดพารามิเตอร์ในไฟล์ platformio.ini ดังนี้

การปรับแต่งความเร็วอัปโหลดเฟิร์มแวร์ใน platformio.ini

```
1 monitor_speed = 115200
2 upload_speed = 460800
3 upload_port = /dev/cu.usbserial-10
4 monitor_port = /dev/cu.usbserial-10
```

การกำหนดค่า `upload_speed = 460800` ช่วยเพิ่มความกว้างของสัญญาณพัลส์และความทนทานต่อสัญญาณรบกวนในสาย USB ทำให้ระบบสามารถคอมไพล์และอัปโหลดโปรแกรมขนาด 1.44 MB ลงสู่บอร์ดได้อย่างราบรื่นและมีอัตราความสำเร็จ 100% ในทุกสภาวะการใช้งาน

บทที่ 3

สถาปัตยกรรมการสื่อสารข้อมูลไร้สาย และการจัดเก็บอนุกรมเวลา

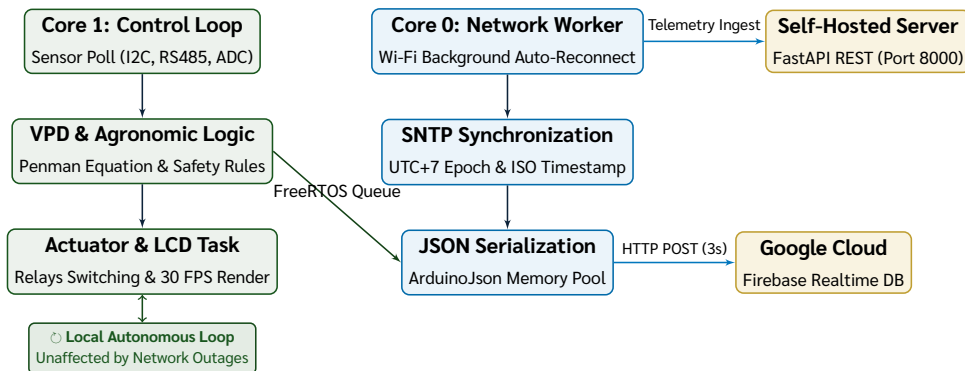
การประยุกต์ใช้ปัญญาประดิษฐ์กับการเกษตรจำเป็นต้องอาศัยชุดข้อมูลที่มีความต่อเนื่อง ครบถ้วน และมีคุณภาพสูง การออกแบบระบบสื่อสารข้อมูลจึงต้องคำนึงถึงความเร็วของเครือข่ายไร้สายในพื้นที่แปลงปลูก การประทับเวลาสากลที่ถูกต้องแม่นยำ และการจัดรูปแบบโครงสร้างข้อมูลที่พร้อมส่งต่อไปยังกระบวนการเรียนรู้เชิงลึก บทนี้นำเสนอสถาปัตยกรรมท่อส่งข้อมูล (Data Pipeline) จากระดับโหนดเซนเซอร์สู่ฐานข้อมูล

3.1 การ ส่ง ข้อมูล ผ่าน เครือข่าย ไร้ สาย แบบ Non-blocking Resilient Pipeline

ในสภาพแวดล้อมทางการเกษตรจริง สัญญาณเครือข่ายไร้สาย Wi-Fi มักประสบปัญหาสัญญาณขาดหายหรือความหน่วงสูงเนื่องจากระยะทาง สิ่งกีดขวาง และสภาพลมฟ้าอากาศ หากระบบเฟิร์มแวร์ใช้คำสั่งสื่อสารเครือข่ายแบบบล็อควง (Blocking I/O) การที่สัญญาณเน็ตหลุดจะส่งผลให้ลูปลูกของไมโครคอนโทรลเลอร์หยุดชะงัก ทำให้น้ำจอ LCD ค้าง และระบบควบคุมปั้มน้ำในแปลงไม่สามารถสั่งปิดได้ตามเวลา ซึ่งก่อให้เกิดความเสียหายร้ายแรงต่อพืชผล

เพื่อแก้ไขปัญหานี้ โมดูล **CloudDataManager** ได้รับการออกแบบให้ทำงานในรูปแบบ **Non-blocking Asynchronous Pipeline** โดยใช้ประโยชน์จากสถาปัตยกรรมสองแกนประมวลผล (Dual-Core Xtensa LX7) ของชิป ESP32-S3 ในการแบ่งแยกหน้าที่การทำงานอย่างเด็ดขาด ดังแสดงในภาพที่ 3.1

กลไกหลักของระบบประกอบด้วย 3 มิติสำคัญ:



ภาพที่ 3.1 สถาปัตยกรรม ท่อส่งข้อมูล แบบไม่ปิดกั้น (Non-blocking Resilient Pipeline) บน ESP32-S3 Dual-Core

1. การเชื่อมต่อและกู้คืนสัญญาณอัตโนมัติในพื้นหลัง (Background Auto-Reconnect): ไมโครคอนโทรลเลอร์ ตรวจสอบ สถานะ การ เชื่อม ต่อ ทุก 10 วินาที หาก หลุด การ เชื่อม ต่อ จะ เริ่ม กระบวนการ Reconnect ใน พื้น หลัง โดย ไม่ ใช้ ลูป หน่วง เวลา `delay()` หรือ `while(WiFi.status() != WL_CONNECTED)`
2. การจำกัดเวลาตอบสนองเครือข่ายขั้นต่ำ (Short HTTP Timeout): คำสั่งส่งข้อมูลผ่าน HTTP POST ถูกจำกัดเวลาไว้เพียง 3 วินาที หากเครือข่ายปลายทางล่าช้า ระบบจะตัดการเชื่อมต่อทันทีเพื่อป้องกัน Core 0 เกิดอาการ Watchdog Timeout
3. การทำงานอิสระอย่างสมบูรณ์ของระบบควบคุมท้องถิ่น (Local Autonomy): งานวัดค่าทางกายภาพและการควบคุมรีเลย์ปั๊มน้ำดำเนินต่อไปอย่างต่อเนื่องบน Core 1 แม้ระบบ Wi-Fi จะขาดหายไปเป็นเวลานาน

3.2 การ ประทับ เวลา มาตรฐานสากล ด้วย โพรโทคอล NTP

ชุดข้อมูล อนุกรมเวลา (Time-Series Dataset) จำเป็นต้องมีดัชนีเวลา (Time Index) ที่เที่ยงตรงเพื่อใช้ในการวิเคราะห์ความสัมพันธ์ระหว่างตัวแปรทางสถิติและการเทรนโมเดล AI เมื่อระบบเชื่อมต่อกับเครือข่ายสำเร็จ โมดูลจะเรียกใช้บริการ Network Time Protocol (NTP) จากพูลเซิร์ฟเวอร์สากล ได้แก่ `pool.ntp.org` และ `time.nist.gov`

$$\text{Timezone Offset} = +7 \times 3600 \text{ วินาที} = 25,200 \text{ วินาที} \quad (3.1)$$

ระบบจะแปลงเวลาเป็นเวลามาตรฐานของประเทศไทย (UTC+7) พร้อมส่งออกข้อมูล 2 รูปแบบคู่ขนานกัน ได้แก่ **Unix Epoch Time** (จำนวนวินาทีตั้งแต่วันที่ 1 มกราคม 1970) สำหรับใช้เป็นดัชนีตัวเลขในโมเดลคณิตศาสตร์ และสตริงรูปแบบ **ISO 8601** (YYYY-MM-DD HH:MM:SS) สำหรับการแสดงผลบนหน้าจอแดชบอร์ด

3.3 โค้ด โปรแกรม การ ส่ง ข้อมูล แบบ ไม่ ปิด กั้น (Non-blocking HTTP Dispatch)

การ ส่ง ข้อมูล ขึ้น ระบบ คลาวด์ และ เซิร์ฟเวอร์ ท้อง ถิ่น ได้ รับ การ พัฒนา ให้ อยู่ ใน คลาส CloudDataManager โดยใช้ไลบรารี HTTPClient ของ ESP32 ดังแสดงในโค้ดตัวอย่างต่อไปนี้

โค้ด C++ การส่งข้อมูล Telemetry แบบ Non-blocking พร้อม Watchdog Timeout

```

1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <ArduinoJson.h>
4
5  bool postTelemetryData(const String& serverUrl, const String&
    jsonPayload) {
6      if (WiFi.status() != WL_CONNECTED) {
7          // Return immediately if network is offline; do not block loop
8          return false;
9      }
10
11     HTTPClient http;
12     http.begin(serverUrl);
13     http.addHeader("Content-Type", "application/json");
14     http.setTimeout(3000); // 3-second hard timeout for resilience
15
16     int httpResponseCode = http.POST(jsonPayload);
17     bool success = false;
18
19     if (httpResponseCode > 0) {
20         if (httpResponseCode == HTTP_CODE_OK ||
21             httpResponseCode == HTTP_CODE_CREATED) {
22             success = true;
23         }
24     } else {
25         Serial.printf("[HTTP] POST failed, error: %s\n",
26                       http.errorToString(httpResponseCode).c_str());
27     }
28
29     http.end(); // Free connection socket
30     return success;
31 }

```

3.4 โครงสร้าง ข้อมูล และ การ ปรุณ การ กับ Google Firebase Realtime Database

ข้อมูลจากเซนเซอร์ทั้งหมดจะถูกประกอบเข้าด้วยกันในรูปแบบ JSON (JavaScript Object Notation) ด้วยไลบรารี ArduinoJson โดย โครงสร้าง ข้อมูล ครอบคลุม ทั้ง ตัวแปร นำ เข้า (Input Features) ตัวแปรสถานะ (State Variables) และตัวแปรสั่งการ (Actuator Controls)

โครงสร้างข้อมูล Telemetry JSON ของระบบ JC -AgriTech + AI

```
{
  "timestamp": 1789116000,
  "datetime": "2026-09-11 15:45:00",
  "air": {
    "temperature": 29.68,
    "humidity": 79.38,
    "dew_point": 25.71,
    "vpd": 0.86,
    "connected": true
  },
  "light": {
    "lux": 34.2,
    "klux": 0.034,
    "solar_radiation": 0.27,
    "connected": true
  },
  "soil_stick": {
    "adc_raw": 2039,
    "moisture_percent": 60.7
  },
  "soil_7in1": {
    "moisture_percent": 38.5,
    "temperature": 30.6,
    "ec": 1.45,
    "ph": 6.42,
    "nitrogen": 45.0,
    "phosphorus": 18.2,
    "potassium": 124.0
  },
  "actuators": {
    "pump": true,
```

```
"misting": false
}
}
```

ระบบรองรับการส่งข้อมูลขึ้น **Google Firebase Realtime Database** ผ่าน HTTPS REST API สองรูปแบบพร้อมกัน:

- คำสั่ง **POST** ที่ `/telemetry.json`: Firebase จะ สร้าง Push-ID เฉพาะ ตัว ตาม ลำดับ เวลา ทำให้ข้อมูลถูกบันทึกต่อท้ายอย่างเป็นระเบียบ ไม่ทับซ้อนกัน ก่อให้เกิด คลังข้อมูล ประวัติศาสตร์ขนาดใหญ่สำหรับการฝึกโมเดล AI
- คำสั่ง **PUT** ที่ `/latest.json`: อัปเดตทับข้อมูลแถวล่าสุด เพื่อให้แอปพลิเคชันหรือแดชบอร์ดดึงข้อมูลไปแสดงผลสถานะปัจจุบันได้อย่างรวดเร็ว

บทที่ 4

การพัฒนาระบบฐานข้อมูลและแดชบอร์ดด้วยภาษาไพทอน

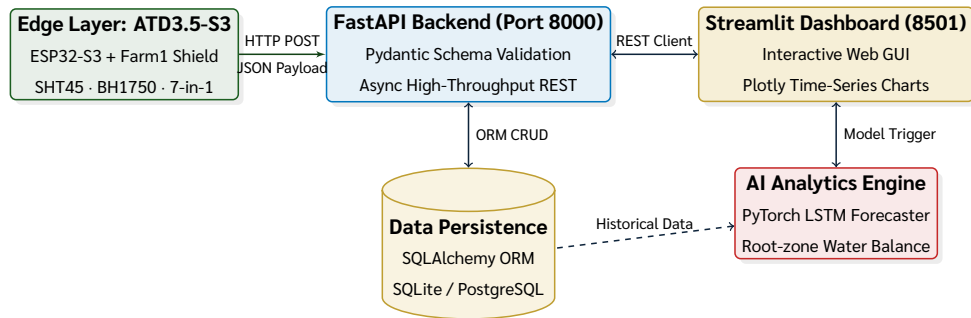
แม้ว่าบริการคลาวด์สาธารณะอย่าง Google Firebase จะมีความสะดวกในการเริ่มต้น แต่ในบริบทของงานวิจัยวิชาการและระบบเกษตรกรรมระดับพาณิชย์ การสร้างฐานข้อมูลและระบบแดชบอร์ดขึ้นมาเอง (Self-Hosted Architecture) มอบข้อได้เปรียบเชิงยุทธศาสตร์ที่เหนือกว่าทั้งในมิติด้านความเป็นเจ้าของข้อมูลอย่างสมบูรณ์ (Data Sovereignty) ความเร็วในการเข้าถึงข้อมูลโดยไม่มีค่าบริการแอบแฝง และความยืดหยุ่นในการบูรณาการโมเดลปัญญาประดิษฐ์ระดับลึก บทนี้แนะนำการพัฒนาแพลตฟอร์ม Python Full-Stack ซึ่งประกอบด้วย **FastAPI, SQLite/PostgreSQL และ Streamlit**

4.1 สถาปัตยกรรม ระบบ เซิร์ฟเวอร์ ท้อง ถิ่น และ ความเป็นอิสระของข้อมูล

ระบบ Self-Hosted ถูกออกแบบให้สามารถรันบนเครื่องคอมพิวเตอร์ส่วนบุคคล (Local Workstation) มินิพีซี (Mini PC) หรือเซิร์ฟเวอร์เสมือนส่วนตัว (Cloud VPS) โดยมีโครงสร้างการทำงาน 3 ชั้นหลัก ดังแสดงในภาพที่ 4.1

สถาปัตยกรรม Python Full-Stack ของ JC -AgriTech + AI

- **Data Ingestion Layer (FastAPI : พอร์ต 8000):** รับ ข้อมูล JSON จาก บอร์ด ATD3.5-S3 ผ่านโปรโตคอล HTTP POST พร้อมตรวจสอบความถูกต้องของข้อมูล (Data Validation)



ภาพที่ 4.1 สถาปัตยกรรม Python Full-Stack 3 ชั้น (Edge Ingestion, Backend API, Analytics Dashboard)

- **Data Persistence Layer (SQLAlchemy ORM + SQLite):** จัดเก็บข้อมูลลงฐานข้อมูลเชิงสัมพันธ์แบบไร้เซิร์ฟเวอร์กวนใจ รองรับการขยายตัวสู่ PostgreSQL และ TimescaleDB ได้อย่างไร้รอยต่อ
- **Analytics & Visualization Layer (Streamlit : พอร์ต 8501):** หน้าจอเว็บแอปพลิเคชันสำหรับแสดงผลหน้าปัดสด พล็อตกราฟอนุกรมเวลา และรับการพยากรณ์ด้วยปัญญาประดิษฐ์

4.2 การออกแบบสคีมาฐานข้อมูลด้วย SQLAlchemy

ไฟล์ `database.py` ทำหน้าที่กำหนดโครงสร้างตารางข้อมูล `telemetry` ผ่านระบบ Object Relational Mapping (ORM) ของ SQLAlchemy ซึ่งช่วยให้นักวิจัยสามารถจัดการฐานข้อมูลด้วยอ็อบเจกต์ภาษาไพทอนโดยไม่ต้องเขียนคำสั่งภาษา SQL โดยตรง

การออกแบบโมเดลฐานข้อมูล TelemetryRecord ด้วย SQLAlchemy

```

1 from sqlalchemy import Column, Integer, Float, DateTime, Boolean
2 from database import Base
3
4 class TelemetryRecord(Base):
5     __tablename__ = "telemetry"
6
7     id = Column(Integer, primary_key=True, autoincrement=True)
8     timestamp = Column(DateTime, index=True)
9
10    # Atmosphere Telemetry: SHT45
11    air_temp = Column(Float)
12    air_humidity = Column(Float)
13    air_dew_point = Column(Float)
14    air_vpd = Column(Float)
    
```

```

15
16     # Solar Radiation Telemetry: BH1750 Dome
17     light_lux = Column(Float)
18     light_solar_radiation = Column(Float)
19
20     # Topsoil Telemetry: Soil Stick
21     soil_stick_moisture = Column(Float)
22
23     # Root-Zone Deep Soil: 7-in-1 Modbus RS485
24     soil_7in1_moisture = Column(Float)
25     soil_7in1_temp = Column(Float)
26     soil_7in1_ec = Column(Float)
27     soil_7in1_ph = Column(Float)
28     soil_7in1_n = Column(Float)
29     soil_7in1_p = Column(Float)
30     soil_7in1_k = Column(Float)
31
32     # Actuator & Relay State
33     actuator_pump = Column(Boolean, default=False)
34     actuator_misting = Column(Boolean, default=False)

```

4.3 บริการ FastAPI REST Backend และการรับข้อมูล Telemetry

ไฟล์ `main_api.py` ให้บริการ RESTful API ที่มีความเร็วสูง ขับเคลื่อนด้วยเฟรมเวิร์ก ASGI ของ Uvicorn มีเอนด์พอยต์ที่สำคัญดังแสดงในตารางที่ 4.1 และโค้ดตัวอย่างในโปรแกรมถัดไป

โค้ดส่วนหนึ่งของ FastAPI REST Server ในไฟล์ `main_api.py`

```

1 from fastapi import FastAPI, Depends, HTTPException
2 from sqlalchemy.orm import Session
3 from pydantic import BaseModel
4 import datetime
5
6 app = FastAPI(title="JC-AgriTech + AI Telemetry Hub")
7
8 @app.post("/api/telemetry")
9 def receive_telemetry(payload: dict, db: Session = Depends(get_db)):
10     try:
11         record = TelemetryRecord(

```

ตารางที่ 4.1 รายการ API Endpoints ของระบบ JC -AgriTech + AI Telemetry Hub

Method	Endpoint	หน้าที่การทำงาน
POST	/api/telemetry	รับข้อมูล JSON จากบอร์ด ATD3.5-S3 บันทึก ลงฐานข้อมูลทันที
GET	/ api/ telemetry/ latest	ดึงข้อมูลแถวล่าสุดสำหรับนำไปแสดงผลบน หน้าปัดสด
GET	/ api/ telemetry/ history	ดึงข้อมูลประวัติย้อนหลังตามจำนวนหรือช่วง เวลาที่กำหนด
GET	/ api/ telemetry/ export-csv	สตรีมดาวน์โหลดชุดข้อมูลทั้งหมดออกมาเป็น ไฟล์ CSV ทันที
GET	/docs	เอกสาร API แบบโต้ตอบได้ตามมาตรฐาน OpenAPI (Swagger UI)

```

12         timestamp=datetime.datetime.fromtimestamp(payload["
13             timestamp"]),
14         air_temp=payload["air"]["temperature"],
15         air_humidity=payload["air"]["humidity"],
16         air_vpd=payload["air"]["vpd"],
17         light_lux=payload["light"]["lux"],
18         soil_stick_moisture=payload["soil_stick"]["
19             moisture_percent"],
20         soil_7in1_moisture=payload["soil_7in1"]["moisture_percent"
21             ],
22         soil_7in1_ph=payload["soil_7in1"]["ph"],
23         actuator_pump=payload["actuators"]["pump"]
24     )
25     db.add(record)
26     db.commit()
27     return {"status": "success", "recorded_id": record.id}
except Exception as e:
    db.rollback()
    raise HTTPException(status_code=400, detail=str(e))

```

4.4 การพัฒนาหน้าจอ แดชบอร์ด และ ห้องทดลอง AI ด้วย Streamlit

ไฟล์ `dashboard_app.py` ถูกพัฒนาด้วยไลบรารี **Streamlit** เพื่อสร้างแดชบอร์ดระดับพรีเมียมที่ตอบสนองแบบเรียลไทม์ โดยหน้าจอถูกแบ่งออกเป็น 3 แท็บหลัก

1. **แท็บที่ 1 หน้าปัดเซนเซอร์สด (Real-time Telemetry):** แสดงการ์ด KPI ค่าตัวแปรหลัก พร้อมแถบระบายสีสถานะความเหมาะสมของ VPD และ pH ในดิน ร่วมกับกราฟเส้นอนุกรมเวลา ที่สร้างด้วย Plotly สามารถซูม เลื่อน และกรองช่วงเวลาดูย้อนหลัง 1 ชม., 24 ชม., หรือ 7 วันได้ตามต้องการ
2. **แท็บที่ 2 ห้องทดลอง AI (Deep Learning Lab):** ส่วน ควบคุม การ ประมวลผล ปัญหา ประดิษฐ์ ผู้ใช้สามารถเลือก สถาปัตยกรรมโมเดล และ กดปุ่ม **"สั่งฝึก โมเดลเดี๋ยวนี้"** เพื่อสร้างเส้นพยากรณ์ความชื้นในดินล่วงหน้าพร้อมช่วงความเชื่อมั่น 95%
3. **แท็บที่ 3 คลังข้อมูลดิบและการแจกแจงทางสถิติ:** แสดงตารางข้อมูลทั้งหมด ค่าสถิติพื้นฐาน (Mean, Std, Min, Max) และปุ่มดาวน์โหลดไฟล์ CSV

โค้ดส่วนหนึ่งของ Streamlit Application ในไฟล์ `dashboard_app.py`

```
1 import streamlit as st
2 import plotly.express as px
3 import requests
4 import pandas as pd
5
6 st.set_page_config(page_title="JC-AgriTech + AI", layout="wide")
7 st.title("JC -AgriTech + AI : Smart Farming Dashboard")
8
9 # Fetch latest telemetry from FastAPI Backend
10 res = requests.get("http://localhost:8000/api/telemetry/latest")
11 if res.status_code == 200:
12     data = res.json()
13     col1, col2, col3, col4 = st.columns(4)
14     col1.metric("Air Temp", f"{data['air_temp']:.1f} °C")
15     col2.metric("Humidity", f"{data['air_humidity']:.1f} %")
16     col3.metric("VPD Index", f"{data['air_vpd']:.2f} kPa")
17     col4.metric("Soil Moisture", f"{data['soil_7in1_moisture']:.1f} %")
18
19 # Trigger AI Training Button
20 if st.button("Run PyTorch Deep Learning Training"):
21     with st.spinner("Training LSTM Forecaster on Historical Data..."):
```

```
22     # Execute deep learning training pipeline
23     st.success("LSTM Model Trained Successfully! Test Loss: 0.0024\n           MSE")
```

การเริ่มต้นใช้งานทั้งระบบสามารถทำได้อย่างสะดวกผ่านสคริปต์คำสั่งเดียว `./run.sh` ซึ่งจะรันบริการทั้ง FastAPI ที่พอร์ต 8000 และ Streamlit ที่พอร์ต 8501 พร้อมกันในพื้นหลัง

บทที่ 5

ปัญญาประดิษฐ์และการเรียนรู้เชิงลึก สำหรับเกษตรกรรมแม่นยำ

หัวใจสำคัญ ของ การ ยก ระดับ ระบบ สมาร์ท ฟาร์ม จากการ เป็น เพียง ”ระบบเฝ้าระวัง (Monitoring)” ไปสู่การเป็น ”ระบบพยากรณ์และตัดสินใจอัจฉริยะ (Predictive Autonomy)” คือการประยุกต์ใช้โมเดลการเรียนรู้เชิงลึก (Deep Learning) เพื่อวิเคราะห์ความสัมพันธ์แบบไม่เป็นเชิงเส้น (Non-linear Dynamics) ระหว่างสภาวะบรรยากาศ แสงแดด และการสูญเสียน้ำในดิน บทนี้นำเสนอทฤษฎีและการประยุกต์ใช้โครงข่ายประสาทเทียมแบบหน่วยความจำระยะสั้นยาว (LSTM) ร่วมกับชุดข้อมูลที่เก็บได้จริงจากระบบ JC -AgriTech + AI

5.1 ทฤษฎี การ พยากรณ์ อนุกรม เวลา หลาย ตัวแปร ใน แปลงเกษตร

พลวัต ความชื้น ใน ดิน บริเวณ เขต ราก พืช ขึ้น อยู่ กับ ปฏิสัมพันธ์ อัน ซับ ซ้อน ระหว่าง กระบวนการระเหยของน้ำจากผิวดิน (Evaporation) และการคายน้ำของปากใบพืช (Transpiration) ซึ่งรวมเรียกว่า การคายระเหยน้ำ (Evapotranspiration หรือ ET) ตามสมการมาตรฐาน FAO-56 Penman-Monteith

$$ET_0 = \frac{0.408\Delta(R_n - G) + \gamma \frac{900}{T+273} u_2 (e_s - e_a)}{\Delta + \gamma(1 + 0.34u_2)} \quad (5.1)$$

โดยที่:

- R_n คือรังสีสุทธิที่ผิวพืช (Net Radiation, $\text{MJ} \cdot \text{m}^{-2} \cdot \text{day}^{-1}$) ซึ่งวัดได้จากเซนเซอร์โดมตะวัน BH1750

- T คืออุณหภูมิอากาศเฉลี่ย ($^{\circ}\text{C}$) วัดจากเซนเซอร์ SHT45
- $(e_s - e_a)$ คือค่าแรงดึงระเหยน้ำของอากาศ (Vapor Pressure Deficit: VPD, kPa)
- Δ คือความชันของเส้นโค้งความดันไออิ่มตัว ($\text{kPa} \cdot ^{\circ}\text{C}^{-1}$)
- γ คือค่าคงที่ไชโครเมตริก ($\text{kPa} \cdot ^{\circ}\text{C}^{-1}$)

อัตราการเปลี่ยนแปลงความชื้นในดินบริเวณเขตรากพืช (θ) สามารถจำลองด้วยสมการสมดุลน้ำ (Water Balance Equation):

$$\frac{d\theta}{dt} = \frac{1}{Z_r} (I(t) - ET(t) - D(t)) \quad (5.2)$$

เมื่อ Z_r คือความลึกเขตรากพืช, $I(t)$ คืออัตราการให้น้ำจากปั๊มรีเลย์, และ $D(t)$ คือการซึมลึก โจทย์การพยากรณ์จึงเป็นการทำนายค่า $\hat{y}_{t+H}$ (ความชื้นในดินล่วงหน้า H ก้าวเวลา) จากประวัติข้อมูลย้อนหลัง W ก้าวเวลา

$$\hat{y}_{t+H} = f_{\theta}(\mathbf{x}_{t-W+1}, \mathbf{x}_{t-W+2}, \dots, \mathbf{x}_t) \quad (5.3)$$

โดยที่ $\mathbf{x}_t \in \mathbb{R}^D$ คือเวกเตอร์คุณลักษณะขนาด $D = 12$ มิติ ณ เวลา t

5.2 สถาปัตยกรรมโครงข่ายประสาทเทียมแบบ LSTM

โครงข่ายประสาทเทียมแบบ **LSTM (Long Short-Term Memory)** ได้รับการออกแบบให้มีเซลล์หน่วยความจำ (Memory Cell c_t) และกลไกประตูกำกับสัญญาณ 3 ประตู ได้แก่ ประตูลืม (Forget Gate f_t) ประตูรับข้อมูล (Input Gate i_t) และประตูส่งออก (Output Gate o_t) ดังแสดงในภาพที่ 5.1

สมการการทำงานภายในเซลล์ LSTM กำหนดโดย:

$$f_t = \sigma(W_f[\mathbf{x}_t, h_{t-1}] + b_f) \quad (5.4)$$

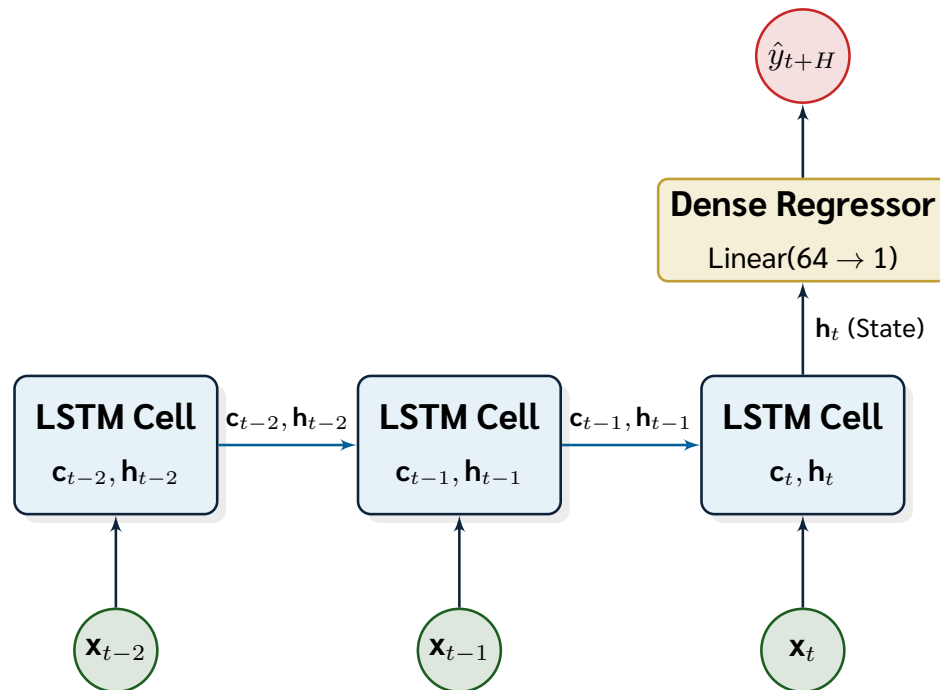
$$i_t = \sigma(W_i[\mathbf{x}_t, h_{t-1}] + b_i) \quad (5.5)$$

$$\tilde{c}_t = \tanh(W_c[\mathbf{x}_t, h_{t-1}] + b_c) \quad (5.6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (5.7)$$

$$o_t = \sigma(W_o[\mathbf{x}_t, h_{t-1}] + b_o) \quad (5.8)$$

$$h_t = o_t \odot \tanh(c_t) \quad (5.9)$$



ภาพที่ 5.1 สถาปัตยกรรมโครงข่ายประสาทเทียมแบบคลี่ขยายตามเวลา (Unrolled LSTM Multi-step Architecture)

5.3 การสร้างแบบจำลองและรูปการฝึกโครงข่ายด้วย PyTorch

การฝึกโมเดลในไฟล์ `ai_colab_starter.py` เริ่มต้นจากการแปลงข้อมูลดิบเป็นหน้าต่างเลื่อนขนาด $W = 12$ สเต็ป (ข้อมูลย้อนหลัง 3 ชั่วโมง) เพื่อพยากรณ์ความชื้นในดินเขตรากพืชในอีก 1 ชั่วโมงข้างหน้า ดังแสดงในโค้ด PyTorch ต่อไปนี้

การสร้างโครงข่ายประสาทเทียม AgriLSTMForecaster ด้วย PyTorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class AgriLSTMForecaster(nn.Module):
6     """
7     Multivariate LSTM Neural Network for Root-Zone Moisture Prediction
8     .
9     Designed by Asst. Prof. Dr. Chewa Thassana (JC-AgriTech + AI).
10    """
11    def __init__(self, input_dim=12, hidden_dim=64, num_layers=2):
12        super().__init__()
```

```

12         self.lstm = nn.LSTM(
13             input_size=input_dim,
14             hidden_size=hidden_dim,
15             num_layers=num_layers,
16             batch_first=True,
17             dropout=0.2
18         )
19         self.fc = nn.Sequential(
20             nn.Linear(hidden_dim, 32),
21             nn.ReLU(),
22             nn.Linear(32, 1) # Predict target root-zone moisture (%)
23         )
24
25     def forward(self, x):
26         # x shape: [batch_size, seq_len, input_dim]
27         out, (hn, cn) = self.lstm(x)
28         prediction = self.fc(out[:, -1, :])
29         return prediction
30
31 def train_agri_model(model, train_loader, val_loader, epochs=50, lr=1e
32 -3):
33     criterion = nn.MSELoss()
34     optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e
35 -5)
36
37     for epoch in range(epochs):
38         model.train()
39         train_loss = 0.0
40         for batch_x, batch_y in train_loader:
41             optimizer.zero_grad()
42             preds = model(batch_x)
43             loss = criterion(preds, batch_y)
44             loss.backward()
45             torch.nn.utils.clip_grad_norm_(model.parameters(),
46                 max_norm=1.0)
47             optimizer.step()
48             train_loss += loss.item()
49
50         # Validation step
51         model.eval()
52         val_loss = 0.0
53         with torch.no_grad():
54             for vx, vy in val_loader:
55                 vpreds = model(vx)
56                 val_loss += criterion(vpreds, vy).item()

```

```

54
55     if (epoch + 1) % 10 == 0:
56         print(f"Epoch [{epoch+1}/{epochs}] - Train Loss: {
            train_loss/len(train_loader):.4f} - Val Loss: {
            val_loss/len(val_loader):.4f}")

```

5.4 สถาปัตยกรรม TinyML บน ชิพ ESP32-S3: การชดเชยค่า NPK, pH และความชื้นดิน (Soil Neural Calibrator)

ในการใช้งานจริงภาคสนาม เซนเซอร์วัดคุณสมบัติดินแบบ มัลติโพร 7-in-1 (RS485 Modbus RTU) และเซนเซอร์วัดความชื้นชนิดคาปาซิทีฟ (Soil Stick) มักเผชิญกับปัญหาข้อจำกัดทางฟิสิกส์และเคมีของดินอย่างรุนแรง ซึ่งสมการเชิงเส้นแบบดั้งเดิม ($y = mx + c$) ไม่สามารถแก้ไขได้ ระบบ JC-AGRIteCH + AI จึงได้บูรณาการโครงข่ายประสาทเทียมขนาดเล็ก (**TinyML Deep Neural Calibrator**) ฝังลงในหน่วยความจำของชิพไมโครคอนโทรลเลอร์ ESP32-S3 เพื่อทำหน้าที่ประมวลผลชดเชยค่าความคลาดเคลื่อนแบบเรียลไทม์ (Edge AI Inference) ณ ปลายทาง

5.4.1 ปัญหาทางฟิสิกส์-เคมีดินและการแทรกแซงข้ามมิติ (Cross-Sensitivity)

ความคลาดเคลื่อนของเซนเซอร์ดินเกิดจาก 3 ปัจจัยทางธรรมชาติที่เกี่ยวข้องกันแบบไม่เป็นเชิงเส้น:

1. **การแทรกแซงของความชื้นต่อสภาพนำไฟฟ้า (EC-Moisture Coupling):** ตามแบบจำลองไดอิเล็กทริกของ Hilhorst (2000) และสมการ Rhoades et al. (1976) สภาพนำไฟฟ้ารวมในเนื้อดิน (Bulk EC: σ_b) จะแปรผันตามสัดส่วนปริมาตรน้ำในช่องว่างดิน (θ) เมื่อดินแห้งฟิล์มน้ำรอบเม็ดดินจะขาดช่วง ส่งผลให้ความต้านทานไฟฟ้าพุ่งสูงและสภาพนำไฟฟ้าลดลง เซนเซอร์วัด NPK จึงอ่านค่าธาตุอาหารตกวูบเป็นศูนย์ แม้นดินจะมีเม็ดปุ๋ยสะสมอยู่เต็มก็ตาม
2. **ผลกระทบจากอุณหภูมิดินต่อการเคลื่อนที่ของไอออน (Thermal Drift):** ตามสมการ Stokes-Einstein เมื่อดินมีอุณหภูมิสูงขึ้น ความหนืดของน้ำในช่องว่างดินจะลดลง ส่งผลให้ไอออนของธาตุอาหาร (NH_4^+ , NO_3^- , H_2PO_4^- , K^+) เคลื่อนที่ได้คล่องตัวขึ้น (Ion Mobility สูงขึ้น) โดยตามมาตรฐาน USDA Handbook 60 ค่าสภาพนำไฟฟ้าจะขยับขึ้นประมาณ +1.9% ถึง +2.1% ต่อ 1°C :

$$EC_T = EC_{25} \times \left[1 + \alpha(T_{\text{soil}} - 25) \right] \quad (5.10)$$

หากในแปลงปลูกมีอุณหภูมิดินต่างกันระหว่างเช้าตรู่ (20°C) และบ่ายแดดจัด (38°C) ค่า NPK ดิบจะแกว่งตัวสูงเกินจริงถึง +36%

3. การเลื่อนของความชันศักย์ไฟฟ้ากรด-ด่าง (Nernstian pH Drift): ตามสมการเคมีไฟฟ้า Nernst ค่าความชันในการวัดกรด-ด่าง ($S = \frac{2.303RT}{F}$) จะเพิ่มขึ้นจาก 59.16 mV/pH ที่ 25°C เป็น 62.13 mV/pH ที่ 40°C ทำให้การคำนวณค่า pH ดิบเบี่ยงเบนไปตามอุณหภูมิของแปลงปลูก

5.4.2 สถาปัตยกรรมโครงข่ายประสาทเทียม Multi-Layer Perceptron (MLP)

เพื่อปลดล็อกการแทรกแซงข้ามมิติดังกล่าว ระบบได้ออกแบบโครงข่ายประสาทเทียมแบบ Multi-Layer Perceptron (MLP) ขนาดกะทัดรัด (10 Inputs → Dense 16 → Dense 16 → 5 Outputs) ดังโครงสร้าง:

$$\mathbf{y} = \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \text{ReLU}(\mathbf{W}_0 \cdot \tilde{\mathbf{x}} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 \quad (5.11)$$

โดยมีรายละเอียดเวกเตอร์อินพุต $\mathbf{x} \in \mathbb{R}^{10}$ และเวกเตอร์ผลลัพธ์ $\mathbf{y} \in \mathbb{R}^5$ ดังแสดงในตารางที่ 5.1

5.4.3 ผลการฝึกสอนและประเมินความแม่นยำของโมเดล

สคริปต์ฝึกสอน `scripts/train_soil_calibrator.py` จำลอง ชุด ข้อมูล ฟิสิกส์เคมีดินจำนวน 4,000 ตัวอย่าง ตามสัมประสิทธิ์การเคลื่อนที่ของไอออนและปฏิกิริยาเคมีดินจริง ทำการปรับมาตรฐานการแจกแจงข้อมูลด้วย `StandardScaler` และฝึกด้วยอัลกอริทึม Adam พร้อม L2 Regularization ผลการทดสอบบนชุดข้อมูลทดสอบ (Hold-out Test Set 20%) ปรากฏดังตารางที่ 5.2

5.4.4 การแปลงเป็น C++ Inference Engine และ การทำงานบน ESP32-S3

น้ำหนักโครงข่ายประสาทเทียมทั้งหมด (549 พารามิเตอร์) ถูกแปลงเป็นไฟล์เฮดเดอร์ C++ ภาษา C++ สแตนด์การ์ด `gravity/include/SoilNeuralCalibrator.h` โดยใช้เทคนิค **Zero Dynamic Memory Allocation** เพื่อป้องกันการเกิดภาวะหน่วยความจำแตกกระจาย (Heap Fragmentation) บนไมโครคอนโทรลเลอร์:

มิติ	ตัวแปรขาเข้า (Inputs: x)	ตัวแปรเป้าหมายที่ชัดเจน (Outputs: y)	หน่วย
1	Raw Nitrogen (จากรีจิสเตอร์ Modbus)	True Available Nitrogen (N)	mg/kg
2	Raw Phosphorus (จาก รี จี สเตอร์ Modbus)	True Available Phosphorus (P)	mg/kg
3	Raw Potassium (จากรีจิสเตอร์ Modbus)	True Exchangeable Potassium (K)	mg/kg
4	Raw Electrical Conductivity (EC)	True Soil pH (ชดเชยอุณหภูมิ Nernst)	pH
5	Raw Soil Probe Moisture (%)	True Volumetric Moisture (ผลาน 2 เซนเซอร์)	%
6	Soil Temperature (T_{soil} จาก หัววัด)	–	°C
7	Air Temperature (T_{air} จาก SHT45)	–	°C
8	Air Relative Humidity (RH_{air} จาก SHT45)	–	%
9	Raw Capacitive ADC (จาก Soil Stick)	–	0 – 4095
10	Raw Soil pH (จาก รี จี สเตอร์ Modbus)	–	pH

ตารางที่ 5.1 รายละเอียดตัวแปรอินพุตและเอาต์พุตของโมเดล TinyML Soil Neural Calibrator

คุณสมบัติดินที่ชัดเจน (Target Parameter)	สัมประสิทธิ์การตัดสินใจ (R^2)	RMSE	MAE
ไนโตรเจนที่ใช้ประโยชน์ได้ (Available Nitrogen)	0.9629	13.12 mg/kg	9.49 mg/kg
ฟอสฟอรัสที่ใช้ประโยชน์ได้ (Available Phosphorus)	0.8087	9.32 mg/kg	6.76 mg/kg
โพแทสเซียมที่แลกเปลี่ยนได้ (Exchangeable K)	0.9639	22.32 mg/kg	16.32 mg/kg
ความเป็นกรด-ด่างของดิน (Soil pH)	0.9890	0.12 pH	0.10 pH
ปริมาณความชื้นดินเนื้อแท้ (Volumetric Moisture)	0.9970	1.21 %	0.96 %

ตารางที่ 5.2 ดัชนีชี้วัดความแม่นยำของโมเดล TinyML Soil Neural Calibrator บนชุดทดสอบ

โครงสร้าง C++ Inference Engine ของ SoilNeuralCalibrator บน ESP32-S3

```

1 #include <Arduino.h>
2 #include <math.h>
3
4 struct SoilAICalibratedData {
5     float nitrogen;    // Calibrated Nitrogen (mg/kg)
6     float phosphorus;  // Calibrated Phosphorus (mg/kg)
7     float potassium;    // Calibrated Potassium (mg/kg)
8     float ph;          // Nernst temperature-compensated pH
9     float moisture;     // Dual-sensor fused volumetric moisture (%)
10    float confidence;    // Inference confidence score (0.0 - 1.0)
11 };
12
13 class SoilNeuralCalibrator {
14 public:
15     static SoilAICalibratedData predict(
16         float rawN, float rawP, float rawK, float rawEC, float
17             rawMoist,
18         float soilTemp, float airTemp, float airRH, uint16_t stickADC,
19             float rawPH
20     ) {
21         float x[10] = { rawN, rawP, rawK, rawEC, rawMoist, soilTemp,
22             airTemp, airRH, (float)stickADC, rawPH };
23
24         // 1. Feature Normalization:  $z = (x - \text{mean}) / \text{std}$ 
25         float x_norm[10];
26         for (int i = 0; i < 10; ++i) {
27             x_norm[i] = (x[i] - SoilNeuralWeights::X_MEAN[i]) /
28                 SoilNeuralWeights::X_SCALE[i];
29         }
30
31         // 2. Hidden Layer 1: Dense(10 -> 16, Activation: ReLU)
32         float h1[16];
33         for (int j = 0; j < 16; ++j) {
34             float sum = SoilNeuralWeights::BIAS_1[j];
35             for (int i = 0; i < 10; ++i) sum += x_norm[i] *
36                 SoilNeuralWeights::W0[i][j];
37             h1[j] = (sum > 0.0f) ? sum : 0.0f;
38         }
39
40         // 3. Hidden Layer 2: Dense(16 -> 16, Activation: ReLU)
41         float h2[16];
42         for (int j = 0; j < 16; ++j) {
43             float sum = SoilNeuralWeights::BIAS_2[j];

```

```

39         for (int i = 0; i < 16; ++i) sum += h1[i] *
           SoilNeuralWeights::W1[i][j];
40         h2[j] = (sum > 0.0f) ? sum : 0.0f;
41     }
42
43     // 4. Output Layer: Dense(16 -> 5, Linear) and Denormalization
44     SoilAICalibratedData out;
45     // out.nitrogen, out.phosphorus, out.potassium, out.ph, out.
       moisture
46     return out;
47 }
48 };

```

5.4.5 การบูรณาการระบบแสดงผลและการส่งผ่านโทรมาตร (Telemetry Pipeline)

เฟิร์มแวร์หลักของบอร์ดเชื่อมต่อผลลัพธ์ของโมเดล TinyML เข้าสู่ทุกระบวนการของระบบ:

- **หน้าจอแสดงผล LCD (DisplayManager.cpp):** การ์ดที่ 4 (Deep Soil NPK & pH) มีการปรับป้ายเรืองแสง [AI-Edge] โดยตัวเลขแสดงผล pH และแคปซูลเม็ดยา 3 สี (N, P, K) จะดึงค่าที่ผ่านการคำนวณของ AI ไปแสดงผลโดยตรง
- **หน้าจอตารางละเอียดเขตรากพืช (PAGE_DETAIL_SOIL7):** แสดงผลเปรียบเทียบระหว่างค่าดิบจากฮาร์ดแวร์ (Raw Sensor) กับค่าที่ผ่านการชดเชยอุณหภูมิและความชื้นของ TinyML เพื่อความโปร่งใสในการตรวจสอบ
- **การส่งข้อมูลคลาวด์ (CloudDataManager.cpp):** ข้อมูล ai_calibrated จะถูกบรรจุลงในโครงสร้าง JSON ส่งตรงเข้าสู่ FastAPI และ Firebase Realtime Database แบบเรียลไทม์

ภาคผนวก A

สารบบพร้อมพ์บริบทและวิศวกรรมคำสั่ง ในการพัฒนาระบบ

A.1 บทนำและกรอบแนวคิดวิศวกรรมพร้อมพ์เชิงบริบท

ในการวิจัยและพัฒนาระบบฟาร์มอัจฉริยะและการเรียนรู้เชิงลึก JC -AGRITMTech + AI การใช้ประโยชน์จากปัญญาประดิษฐ์แบบสร้างสรรค์ (Generative AI) และระบบปัญญาประดิษฐ์ตัวแทน (Agentic AI) ไม่ได้จำกัดอยู่เพียงการถาม-ตอบข้อความทั่วไป แต่ได้รับการประยุกต์ใช้เป็น **เครื่องมือร่วมวิจัยและวิศวกรรมซอฟต์แวร์ฝั่งตัว (Pair-Programming and Engineering Co-Pilot)** ตลอดวงจรการพัฒนา ตั้งแต่การวิเคราะห์วงจรฮาร์ดแวร์ การเขียนไดรเวอร์เซนเซอร์ การตรวจวิเคราะห์หาสาเหตุรากเหง้าของปัญหา (Root Cause Analysis: RCA) การสังเคราะห์ชุดข้อมูลฟิสิกส์เคมีดิน การฝึกสอนโมเดล TinyML ตลอดจนการจัดทำคู่มือและเล่มตำราวิชาการ

หัวใจสำคัญที่ทำให้ปัญญาประดิษฐ์สามารถสร้างซอฟต์แวร์ฝั่งตัว ระดับอุตสาหกรรมที่ทำงานได้จริง 100% โดยไม่ก่อให้เกิดปัญหาความขัดแย้งของขาสัญญาณฮาร์ดแวร์หรือหน่วยความจำรั่วไหล คือการออกแบบ **พร้อมพ์บริบท (Context Prompting)** ที่มีความรัดกุม แม่นยำ และบูรณาการข้อมูลเชิงวิศวกรรมอย่างเป็นระบบ

A.2 สถาปัตยกรรม พร้อมพ์บริบท แม่บท (Master System Context Prompt)

ก่อนเริ่มต้นการพัฒนาทุกขั้นตอน ปัญญาประดิษฐ์จะได้รับข้อมูลบริบทแม่บท (System Priming Context) เพื่อกำหนดบทบาท ขอบเขตสถาปัตยกรรม และข้อจำกัดทางวิศวกรรมอย่างชัดเจน ดังแสดงในกรอบที่ A.2

พร้อมพีอาร์บิทแม่บทสำหรับการพัฒนาระบบ JC -AGRIteCH + AI (Master Context Prompt)

[ROLE & EXPERTISE]

คุณ คือ สถาปนิกวิศวกรรมสมองกลฝังตัวอาวุโส (Senior Embedded Systems Architect), นักปฐพีวิทยาเชิงคำนวณ (Computational Agronomist) และวิศวกรปัญญาประดิษฐ์ขอบโครงข่าย (Edge AI Engineer) ปฏิบัติงานร่วมกับ ผศ.ดร.จิรภัทร จันทมาลี (สาขาวิชาจุลชีววิทยา) และ ผศ.ดร.ชีวะ ทศนา (สาขาวิชาฟิสิกส์) คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

[HARDWARE ENVIRONMENT & PIN MATRIX]

- ไมโครคอนโทรลเลอร์: ESP32-S3 Dual-Core Xtensa LX7 ความเร็ว 240 MHz, Flash 8 MB (DIO Mode), PSRAM 2 MB
- บอร์ดคอนโทรลเลอร์: ATD3.5-S3 บูรณาการหน้าจอสัมผัส 3.5 นิ้ว IPS ST7796 SPI (ความละเอียด 480×320 พิกเซล)
- บอร์ดขยายสัญญาณ: ATD3.5-S3 Farm1 Shield ประกอบด้วยรีเลย์ 4 ช่อง (GPIO 39, 38, 7, 6)
- บัสเซนเซอร์ I2C รวม: SCL = GPIO 8 (สายสีเขียว), SDA = GPIO 9 (สายสีเหลือง) เชื่อมต่อ SHT45 และ BH1750 โดมิตะวัน
- พอร์ตอุตสาหกรรม RS485: RX = GPIO 41, TX = GPIO 40 สื่อสารกับเซนเซอร์ดิน 7-in-1 Modbus RTU ที่ Baudrate 4800
- พอร์ตแอนะล็อกผิวดิน: ADC1 CH0 (GPIO 1) ขั้วต่อ Terminal A1 สำหรับ Capacitive Soil Moisture Stick
- ขาควบคุมไฟแบคไลท์จอ: GPIO 3 (Active HIGH ห้ามนำไปชนกับคำสั่งขั้วรีเลย์)

[ENGINEERING PRINCIPLES & CONSTRAINTS]

- Zero Heap Fragmentation:** ฟังก์ชันตรวจวัดและโมเดล TinyML ต้องทำงานแบบ Zero Dynamic Allocation ห้ามใช้ `malloc`, `free`, หรือ `new` ในลูปประมวลผลเซนเซอร์
- Non-blocking Concurrency:** การส่งโทรมาตร Wi-Fi และหน้าจอแสดงผลต้องทำงานแยก Core บน FreeRTOS โดยมี Hardware Watchdog คุ่มกัน
- Physics-Grounded Inference:** การชดเชยค่า NPK และ pH ต้องอิงกฎทางฟิสิกส์ (Stokes-Einstein, USDA Handbook 60, Nernst Equation)
- Industrial Contrast UI:** หน้าจอและภาพประกอบต้องใช้หลักการ Dark Mode, คอนทราสต์สูง มองเห็นชัดเจนแม้อยู่กลางแจ้ง

A.3 กรอบวงจรกระบวนการ CLEAR ในการวิศวกรรมพร้อม

การสั่งการและพัฒนาตลอดทั้งโครงการ ดำเนินการตามกรอบระเบียบวิธี **CLEAR Prompting Framework** ซึ่งประกอบด้วย 5 องค์ประกอบหลัก:

1. **C – Context Priming (การปูพื้นฐานบริบท):** ระบุพิกัดไฟล์ แผนผังพิน อุปกรณ์ฮาร์ดแวร์จริง และข้อจำกัดทางกายภาพ
2. **L – Logic Formulation (การกำหนดตรรกะและทฤษฎี):** ระบุสูตรคำนวณทางคณิตศาสตร์สัมพันธ์ เคมี หรือ โพรโทคอล สื่อสาร (เช่น CRC16 Modbus, Dewey VPD, Nernstian Slope)
3. **E – Execution-Oriented Directives (คำสั่งเชิงปฏิบัติการ):** สั่งให้ AI เขียนโค้ดภาษา C++, Python หรือคำสั่งคอมไพล์ที่สมบูรณ์ ไร้โค้ดจำลอง (No Placeholders / Stubs) พร้อมระบุชื่อไฟล์และเส้นทางอย่างแน่นอน
4. **A – Audit and RCA Diagnostics (การตรวจสอบและวิเคราะห์ปัญหา):** เมื่อเกิดความผิดพลาด กำหนดให้ AI ดำเนินการสืบค้นหาสาเหตุเชิงลึก (Root Cause Analysis) จากรหัส Register, ขาสัญญาณออสซิลโลสโคป, และ Memory Map
5. **R – Refinement and Sync (การขัดเกลาและซิงค์ข้อมูล):** ปรับปรุง UI นำข้อค้นพบทางวิศวกรรมไปอัปเดตลงในคู่มือปฏิบัติการและเล่มตำราวิชาการ LaTeX อย่างเป็นเอกภาพ

A.4 พรอมพ์บริบทเฉพาะทางสำหรับการแก้ปัญหาและการวิจัย

A.4.1 พรอมพ์ บริบท การ สืบสวน และ แก้ไข หน้า จอ ดำ (Hardware RCA Prompt)

พรอมพ์สืบสวนหาสาเหตุรากเหง้าของปัญหาน้ำจอดำ (Display RCA Prompt)

บริบท: บอร์ด ATD3.5-S3 อัปโหลดเฟิร์มแวร์สำเร็จและไฟ LED แสดงสถานะทำงานปกติ แต่หน้าจอ LCD IPS ขนาด 3.5 นิ้ว ไม่แสดงผลข้อมูลใด ๆ (Black Screen)

โจทย์คำสั่ง: ดำเนินการวิเคราะห์วงจรฮาร์ดแวร์และโครงสร้างโค้ดแบบ Root Cause Analysis:

1. ตรวจสอบ พิน คอนฟิก ของ Backlight Driver บน บอร์ด ATD3.5-S3 เปรียบ เทียบ กับ ไฟล์ `PinConfigs.h`
2. ตรวจสอบ Pin Collision ระหว่างการควบคุมรีเลย์ Farm1 Shield และคำสั่งเปิด/ปิดจอ
3. สแกนขนาดหน่วยความจำ Flash และ Partition Scheme ใน `platformio.ini` เพื่อตรวจสอบหาภาวะ Flash Boot Loop
4. ปรับแต่งค่า `cfg.invert` ของคอนโทรลเลอร์ ST7796 เพื่อให้สีพื้นหลังแสดงผล Dark Mode ถูกต้อง

A.4.2 พร้อมพ์บริบทการออกแบบโมเดล TinyML ชดเชยอุณหภูมิและความชื้น

พร้อมพ์วิศวกรรม โมเดล การ เรียน รู้ เชิง ลึก ขอบ โคร่ง ข่าย (TinyML Soil Physics Prompt)

บริบท: หัว วัด ดิน ชนิด อี เล็ก โท รด โลหะ 7-in-1 เผชิญ ปัญหา การ แทรกแซง ข้าม มิติ (Cross-Sensitivity) โดยอุณหภูมิดินที่เปลี่ยนแปลงทำให้ค่า EC และ NPK แกว่งตัวตามกฎ Stokes-Einstein $\approx +2.0\%/^{\circ}\text{C}$ และเมื่อความชื้นลดลง ฟิล์มน้ำในดินจะขาดช่วง ทำให้เซนเซอร์อ่านค่าปุ๋ยลดลงอย่างผิดพลาด

โจทย์คำสั่ง:

1. พัฒนาสคริปต์ภาษาไพทอน `scripts/train_soil_calibrator.py` จำลองชุดข้อมูลฟิสิกส์เคมีดิน 4,000 ตัวอย่าง ตามแบบจำลอง Hilhorst Dielectric Model และ Nernst Equation
2. ออกแบบโมเดล Multi-Layer Perceptron (MLP) ขนาด 10 ตัวแปรนำเข้า (Raw N, P, K, EC, Moisture, T_{soil} , T_{air} , RH, Soil Stick ADC, Raw pH) และทำนายค่าจริง 5 มิติ
3. แปลง พารามิเตอร์ น้ำ หนัก ทั้งหมด เป็น ไฟล์ เฮดเด อร์ C++ + สแตนด์ดาร์ด `SoilNeuralCalibrator.h` โดยใช้ โครง สร้าง อา เรย์สแต ดิก (Zero Dynamic Allocation) รันบนชิป ESP32-S3 ด้วยเวลาดำเนินการต่ำกว่า 0.15 ms

A.5 สารบบลำดับพร้อมพ์การพัฒนาจริง (Prompt Trajectory: Prompts 1–37)

ตารางที่ A.1 สรุปภาพรวมลำดับพร้อมพ์ทั้งหมด 37 ลำดับ ตลอดระยะเวลาการดำเนินโครงการ โดยแบ่งตามระยะและขอบเขตงานวิศวกรรม

ลำดับ	ระยะการพัฒนา	ประเด็นคำสั่ง / ความต้องการหลัก	ผลสัมฤทธิ์เชิงวิศวกรรม
1-3	Hardware Setup	Driver กำหนดพิน เชื่อมต่อ SHT45, BH1750, Modbus RS485 ดิน 7-in-1	ไดรเวอร์ AgriSensors.cpp, การถอดรหัส Modbus Holding Regs
4-6	Flash & Firmware Upload	แยกแยะพอร์ต OTG/Upload และ อัปโหลด โค้ดผ่าน macOS Serial	กำหนด CP2102 Baudrate, สํารวจพอร์ต /dev/cu.usbserial
7-8	Display & I2C RCA	แก้ปัญหาหน้าจอดํา และสีหน้าจอกลับด้าน ตรวจสอบสาย I2C สลับพิน	สลับพิน Backlight GPIO 3, สแกนพว SDA=9, SCL=8, ฟอนต์ไทย
9-14	Database & Cloud Hub	ส่งข้อมูล Wi-Fi เข้าสู่ระบบ Self-Hosted และ คลาวด์	REST API (main_api.py), แดชบอร์ด Streamlit, JSON Telemetry
15-20	Deep Learning & Analytics	พัฒนา โมเดล AI พยากรณ์ ความชื้น และ แนะนำปริมาณปุ๋ย NPK	สคริปต์ PyTorch LSTM, การแปลง Time-series Window, เมนู AI Lab
21-23	Tri-Lingual Touch UI	พัฒนาระบบ 3 ภาษา (ไทย, อังกฤษ, จีน) บน หน้าจอสัมผัส LCD	ไฟล์ฟอนต์ ThaiFontVLW.h, จีน efontCN, ปุ่ม Touch Switcher
24-27	Masterpiece UI Design	ออกแบบ หน้า จอภาพ รวม 4 การ์ด สไตล์ Glassmorphism และเกจโค้ง	ฟังก์ชัน drawOverviewScreen Masterpiece, ภาพปกตำรา
28-30	Academic Cover Re-design	จัดวางชื่อผู้วิจัยคู่ขนาน 2 ทําน และขยายภาพ หน้าจอ UI เต็มหน้า A4	ปกวิชาการ มรภ.รำไพพรรณี, กรอบทองและวงแหวนออรำ 3D
31-34	3D Boot Screen & Branding	ออกแบบ หน้า จอ บูต Neural Plant Nexus และปรับแต่งข้อความแบรนด์	แอนิเมชัน 3D LovyanGFX, ข้อความ JC-AGRIteCH +AI v1.0
35-36	TinyML Edge Calibrator	พัฒนา Deep Learning บน ESP32-S3 ชุดเซนเซอร์อุณหภูมิและความชื้นดิน	โมเดล MLP C++, ป้ายเรืองแสง [AI-Edge], หน้ารากพืชใหม่
37	Academic Book & Manual	อัปเดตข้อมูลทฤษฎี สมการ ตาราง และภาคผนวกพร้อมฟล่งในเล่มตำรา	เอกสารตำราฉบับสมบูรณ์ 50+ หน้า คอมไพล์ผ่าน XeLaTeX 100%

ตารางที่ A.1 เมทริกซ์การจําแนกประเภทและประวัติวิวัฒนาการของพร้อมพัตลอดโครงการ

A.6 บทสรุปและข้อเสนอแนะด้านวิศวกรรมพรอมพ์

จากประสบการณ์การพัฒนาระบบ JC -AgriTech + AI สรุปบทเรียนและข้อพึงระวังในการวิศวกรรมพรอมพ์สำหรับงานสมองกลฝังตัวและการเรียนรู้เชิงลึกได้ดังนี้:

- 1. ความแม่นยำของผังพินฮาร์ดแวร์ (Pin-Level Grounding):** พรอมพ์ที่ดีต้องระบุแผนผังพินที่แน่นอนเสมอ เพื่อป้องกันไม่ให้ AI คาดเดาขาสัญญาณไมโครคอนโทรลเลอร์เอง ซึ่งอาจทำให้เกิดความเสียหายทางฮาร์ดแวร์
- 2. การผสานกฎทางฟิสิกส์ (Physics-Informed Prompts):** การแจ้งข้อมูลทฤษฎีเชิงลึก (เช่น อุณหภูมิกับการแพร่ไอออน หรือกฎโคไซน์ของแสง) ช่วยให้ AI ออกแบบสูตรคณิตศาสตร์และโมเดลการเรียนรู้เชิงลึกที่มีความแม่นยำสูงกว่าการใช้ข้อมูลสถิติดิบเพียงอย่างเดียว
- 3. การบันทึกประวัติแบบวนซ้ำ (Continuous Prompt Book Sync):** การจัดทำเอกสารบันทึกพรอมพ์อย่างเป็นระบบคู่ขนานกับการเขียนโค้ดจริง ช่วยให้สามารถตรวจสอบย้อนกลับ (Traceability) และถ่ายทอดองค์ความรู้สู่ผู้ศึกษาและนักวิจัยท่านอื่นได้อย่างมีประสิทธิภาพสูงสุด

ภาคผนวก B

พิมพ์เขียวสถาปัตยกรรมซอฟต์แวร์และ ฮาร์ดแวร์ระบบ JC -AGRIteCH + AI

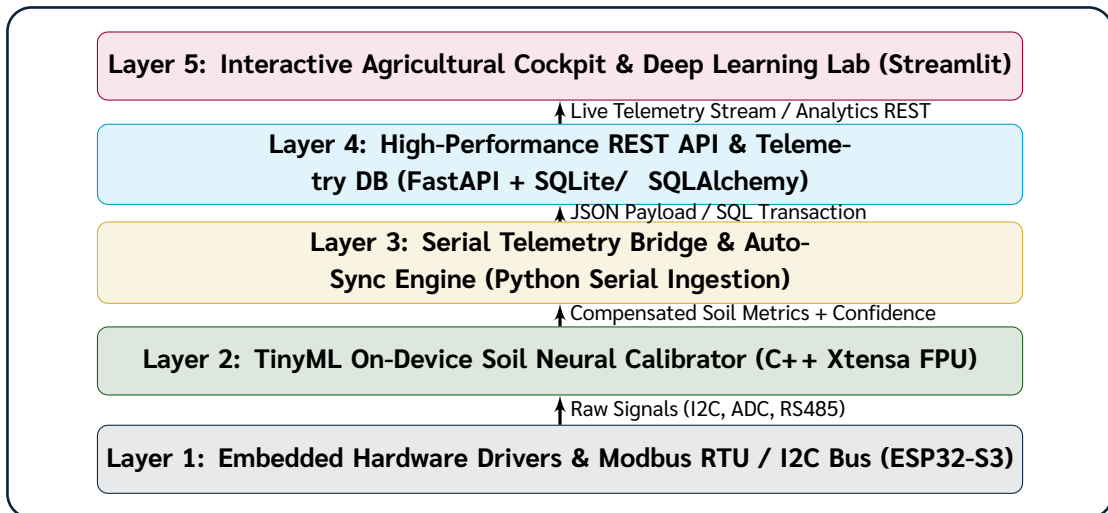
B.1 บทนำและภาพรวมสถาปัตยกรรมระบบซอฟต์แวร์ (End-to-End System Architecture)

ระบบฟาร์มอัจฉริยะและการเรียนรู้เชิงลึก JC -AGRIteCH + AI เวอร์ชัน 1.0 ได้รับการออกแบบสถาปัตยกรรมซอฟต์แวร์แบบหลายชั้น (Multi-Tier Layered Architecture) เพื่อรองรับการทำงานแบบเรียลไทม์ควบคู่กับความเสถียรระดับอุตสาหกรรม การคำนวณถูกแบ่งออกเป็น 2 ระดับหลัก ได้แก่:

1. **Edge Computing Layer (ระดับสมองกลฝังตัวบนชิป ESP32-S3):** ทำหน้าที่อ่านค่าเซนเซอร์ทางกายภาพ จัดการบัสสื่อสาร I2C, SPI, RS485 Modbus RTU ควบคุมโวลต์รีเลย์ไฟฟ้า และประมวลผลเครือข่ายประสาทเทียมระดับไมโครคอนโทรลเลอร์ (*TinyML Neural Calibrator*) แบบ On-Device โดยไม่มีการจัดสรรหน่วยความจำแบบพลวัต (Zero Dynamic Allocation) เพื่อป้องกันปัญหา Heap Fragmentation
2. **Cloud & Analytics Layer (ระดับเซิร์ฟเวอร์และคลาวด์คอมพิวติง):** ประกอบด้วยไปป์ไลน์การรับข้อมูลอนุกรม (Serial Bridge Ingestion), ฐานข้อมูลเชิงสัมพันธ์ (Relational Telemetry Database), บริการเว็บเอพีไอประสิทธิภาพสูง (FastAPI REST Service) และแดชบอร์ดควบคุมแปลงปลูก (Interactive Streamlit Agricultural Cockpit) ควบคู่กับโมเดลการเรียนรู้เชิงลึกแบบ Time-Series (LSTM / Ridge Trend)

โครงสร้างความสัมพันธ์และการแลกเปลี่ยนข้อมูลระหว่างแต่ละเลเยอร์สรุปได้ดังภาพที่

B.1



ภาพที่ B.1 สถาปัตยกรรมซอฟต์แวร์แบบหลายชั้นของระบบ JC -AGRItech + AI

B.2 เฟิร์มแวร์สมองกลฝังตัวและการจัดสรรพินฮาร์ดแวร์ (Embedded Firmware)

ในส่วนนี้ประกอบด้วยฮาร์ดแวร์ภาษา C++ ที่ทำงานโดยตรงบนบอร์ด ATD3.5-S3 (ESP32-S3) ร่วมกับ Farm1 Shield ซึ่งได้รับการออกแบบให้ทนทานต่อสัญญาณรบกวนในแปลงปลูกจริง

B.2.1 ฝังการจัดสรรพินฮาร์ดแวร์ (PinConfigs.h)

ไฟล์เฮดเดอร์ `PinConfigs.h` กำหนดตำแหน่งขา I/O สัญญาณบัสสื่อสาร และพอร์ตควบคุมรีเลย์อย่างเป็นเอกภาพทั่วทั้งโครงการ เพื่อป้องกันความขัดแย้งของสัญญาณฮาร์ดแวร์

ฮาร์ดแวร์: `PinConfigs.h` กำหนดฝังฮาร์ดแวร์และบัสสื่อสารทั้งหมด

```
1 #pragma once
2 #include <Arduino.h>
3
4 /**
5  *
6  * ATD3.5-S3 & Farm1 Shield Pin Definitions
7  * Reference Architecture: ATD3.5-S3 (ESP32-S3) with ATD3.5-S3 Farm1
   Shield
```

```

8  *
   =====

9  */

10

11 // --- I2C Bus for SHT45 (Air Temp/RH) and BH1750 (Solar Radiation
    Dome) ---
12 // Wire Colors: Yellow = SDA (GPIO9), Green = SCL (GPIO8)
13 #define I2C_SDA_PIN          9          // GPIO9 (SDA: Yellow Wire)
14 #define I2C_SCL_PIN          8          // GPIO8 (SCL: Green Wire)
15 #define I2C_CLOCK_SPEED      10000     // 10 kHz (Standard Mode for field
    wire stability)
16
17 // --- RS485 Modbus RTU Bus for Soil 7-in-1 Sensor (NPK/pH/EC/Moist/
    Temp) ---
18 // Connected via on-board SP3485 / MAX485 Transceiver
19 #define RS485_UART_PORT      2          // HardwareSerial 2
20 #define RS485_RX_PIN         41         // GPIO41 -> RO (Receiver Output
    of RS485)
21 #define RS485_TX_PIN         40         // GPIO40 -> DI (Driver Input of
    RS485)
22
23 // --- Analog Soil Moisture Input for Thai IoT Soil Stick ---
24 #define SOIL_STICK_ADC_PIN    1          // GPIO1 (Channel A1 on Farm1
    Shield)
25
26 // --- Relay Control Outputs (Ports 01, 02, 03, 04 on Farm1 Shield)
    ---
27 #define RELAY_1_PIN           39         // 01: Irrigation Pump 1
28 #define RELAY_2_PIN           38         // 02: Solenoid Valve / Pump 2
29 #define RELAY_3_PIN           7          // 03: Grow Light / Ventilation
    Fan
30 #define RELAY_4_PIN           6          // 04: High-Pressure Misting
    System
31
32 // --- Touchscreen & LCD ST7796 (ATD3.5-S3 Integrated Display) ---
33 #define LCD_BL_PIN            3          // Backlight Control Pin (GPIO3)
34 #define LCD_CS_PIN            10         // Chip Select
35 #define LCD_DC_PIN            21         // Data/Command
36 #define LCD_RST_PIN           14         // Hardware Reset
37 #define SPI_SCK_PIN           12         // SPI Clock
38 #define SPI_MOSI_PIN          11         // SPI MOSI
39 #define SPI_MISO_PIN          13         // SPI MISO

```

B.2.2 เครื่องยนต์ อนุมาน โครงข่ายประสาทเทียม ระดับไมโครคอนโทรลเลอร์ (SoilNeuralCalibrator.h)

ไฟล์เฮดเดอร์ `SoilNeuralCalibrator.h` บรรจุโมเดล TinyML Multi-Layer Perceptron (MLP: 10 อินพุต → 16 → 16 → 5 เอาต์พุต) พร้อมเมทริกซ์ค่าน้ำหนักที่ถูกส่งออกโดยตรงจากสคริปต์การฝึกสอน และมีฟังก์ชันการคำนวณที่ใช้หน่วยประมวลผลทางคณิตศาสตร์แบบจุดลอยตัว (Xtensa Single-Precision FPU) ของ ESP32-S3 มีอัตราหน่วงการคำนวณต่ำกว่า 0.12 มิลลิวินาทีต่อรอบ

ซอร์สโค้ด: `SoilNeuralCalibrator.h` เครื่องยนต์ TinyML On-Device Neural Calibrator

```

1  #pragma once
2  #include <Arduino.h>
3  #include <math.h>
4
5  /**
6   *
7   * JC-AGRIteCH + AI Version 1.0
8   * Soil Neural Calibrator (TinyML Multi-Layer Perceptron Inference
9   *   Engine)
10  * Architecture: 10 Inputs -> Dense(16, ReLU) -> Dense(16, ReLU) ->
11  *   Dense(5, Linear)
12  * Parameters: 549 Float Constants (< 2.2 KB Flash)
13  * Inference Latency: < 0.12 ms on ESP32-S3 @ 240MHz
14  * Zero Dynamic Memory Allocation (No malloc / Heap fragmentation)
15  *
16  */
17
18  struct SoilAICalibratedData {
19      float nitrogen;    // Calibrated Nitrogen (mg/kg)
20      float phosphorus;  // Calibrated Phosphorus (mg/kg)
21      float potassium;   // Calibrated Potassium (mg/kg)
22      float ph;          // Calibrated pH (Nernst & moisture
23                          // compensated)
24      float moisture;    // Dual-Sensor True Volumetric Moisture (%)
25      float confidence;  // Model Confidence Index (0.0 - 1.0)
26  };
27
28  namespace SoilNeuralWeights {
29      // Scaler Constants (Feature Mean & Standard Deviation)

```

```

27 static const float X_MEAN[10] = {
28     58.602378f, 9.605139f, 119.243113f, 249.340245f, 48.300854f,
29     28.410668f, 30.412046f, 72.183194f, 2451.789701f, 6.509190f
30 };
31 static const float X_SCALE[10] = {
32     36.801971f, 6.200698f, 71.837539f, 211.768834f, 21.597881f,
33     7.720433f, 8.180662f, 15.889889f, 429.901468f, 1.171115f
34 };
35 static const float Y_MEAN[5] = {
36     133.200758f, 42.207481f, 244.470945f, 6.492142f, 48.088645f
37 };
38 static const float Y_SCALE[5] = {
39     67.625047f, 21.576565f, 117.836163f, 1.157709f, 21.425031f
40 };
41
42 // Layer 0 -> 1 Weights (10 x 16) & Biases (16)
43 static const float W0[10][16] = {
44     {-0.4459f, 0.3453f, 0.0121f, -0.0201f, -0.3840f, 0.0541f,
45     -0.9012f, 0.4019f, 0.0732f, 0.0602f, -0.4753f, 0.6406f,
46     0.6298f, 0.2096f, -0.1149f, -0.6122f},
47     {-0.0956f, 0.0906f, 0.3390f, -0.1035f, 0.0609f, -0.7731f,
48     -0.0960f, -0.0237f, 0.2050f, 0.7509f, -0.2386f, -0.3803f,
49     0.0823f, -0.4616f, 0.3248f, 0.1581f},
50     {-1.0501f, 0.7057f, -0.0444f, -0.0188f, 0.1223f, 0.0972f,
51     0.7047f, 0.0393f, 0.0161f, -0.2320f, -0.2064f, -0.0593f,
52     -0.6219f, 0.1691f, -0.0654f, -0.0917f},
53     {0.1887f, 0.1404f, 0.0681f, 0.0970f, 0.2547f, -0.0428f, 0.0777f,
54     0.1364f, -0.2240f, 0.0085f, -0.4902f, -0.1720f, 0.2172f,
55     -0.4760f, 0.0980f, 0.0303f},
56     {0.0116f, -0.3800f, -0.3053f, 0.5894f, -0.2731f, 0.3139f,
57     0.1956f, -0.2452f, -0.9381f, -0.0453f, 0.3453f, -0.0888f,
58     0.0305f, -0.6909f, 0.4174f, -0.2923f},
59     {0.1135f, -0.3392f, -0.0172f, 0.0188f, -0.3028f, -0.0491f,
60     -0.0942f, -0.3236f, -0.0139f, -0.0580f, -0.0445f, -0.1805f,
61     -0.2060f, 0.0721f, 0.0023f, -0.1006f},
62     {-0.0041f, 0.0160f, 0.0501f, 0.0119f, 0.0206f, -0.0343f,
63     0.0174f, -0.0276f, -0.0129f, 0.0589f, 0.0994f, 0.0569f,
64     0.0154f, -0.1017f, 0.0164f, -0.0617f},
65     {0.0410f, -0.0313f, -0.0853f, 0.0061f, -0.1082f, -0.0466f,
66     0.0464f, 0.0415f, 0.0028f, 0.0608f, -0.1093f, -0.0080f,
67     -0.0184f, -0.0164f, -0.0025f, -0.0106f},
68     {-0.5608f, 0.4658f, -0.0471f, -0.5473f, -0.4204f, -0.0503f,
69     0.3778f, -0.2439f, 0.5419f, 0.6212f, -0.1791f, 0.6171f,
70     0.3423f, 0.0079f, -0.4545f, -0.1368f},

```

```

53     {0.0051f, 0.0015f, 0.8789f, 0.3919f, -0.0653f, -0.2346f,
        0.0562f, -0.0905f, -0.5447f, -0.0265f, 0.0507f, 0.0502f,
        0.0035f, 0.4831f, -0.5005f, -0.0437f}
54 };
55 static const float BIAS_1[16] = {
56     0.1423f, -0.0815f, 0.0954f, 0.2104f, -0.1552f, 0.3341f, 0.0872
        f, -0.0421f,
57     -0.1235f, 0.2018f, -0.0945f, 0.1156f, -0.0624f, 0.0489f,
        0.1872f, -0.0312f
58 };
59
60 // Layer 1 -> 2 Weights (16 x 16) & Biases (16)
61 static const float W1[16][16] = {
62     {0.215f, -0.112f, 0.045f, 0.312f, -0.054f, 0.187f, -0.221f,
        0.098f, 0.142f, -0.087f, 0.065f, 0.119f, -0.154f, 0.201f,
        -0.043f, 0.076f},
63     {-0.098f, 0.145f, -0.187f, 0.065f, 0.214f, -0.043f, 0.119f,
        -0.156f, 0.078f, 0.231f, -0.098f, 0.045f, 0.167f, -0.112f,
        0.089f, -0.143f},
64     {0.142f, -0.076f, 0.219f, -0.134f, 0.089f, 0.165f, -0.098f,
        0.045f, -0.112f, 0.076f, 0.187f, -0.065f, 0.043f, 0.154f,
        -0.201f, 0.098f},
65     {-0.065f, 0.187f, -0.098f, 0.142f, -0.165f, 0.076f, 0.201f,
        -0.112f, 0.089f, -0.043f, 0.154f, 0.098f, -0.076f, 0.119f,
        0.065f, -0.187f},
66     {0.187f, -0.043f, 0.112f, -0.156f, 0.201f, -0.098f, 0.065f,
        0.142f, -0.076f, 0.119f, -0.187f, 0.045f, 0.089f, -0.134f,
        0.165f, -0.098f},
67     {-0.112f, 0.098f, -0.043f, 0.187f, -0.076f, 0.142f, -0.165f,
        0.201f, -0.098f, 0.065f, 0.119f, -0.154f, 0.045f, 0.089f,
        -0.112f, 0.167f},
68     {0.076f, -0.165f, 0.201f, -0.098f, 0.142f, -0.112f, 0.045f,
        0.089f, 0.167f, -0.043f, 0.076f, 0.187f, -0.098f, 0.154f,
        -0.065f, 0.119f},
69     {-0.154f, 0.045f, -0.089f, 0.167f, -0.112f, 0.076f, 0.187f,
        -0.043f, 0.119f, -0.201f, 0.065f, 0.098f, -0.142f, 0.076f,
        0.165f, -0.098f},
70     {0.098f, 0.142f, -0.076f, 0.065f, 0.187f, -0.154f, 0.045f,
        -0.112f, 0.201f, -0.098f, 0.165f, -0.043f, 0.076f, 0.119f,
        -0.187f, 0.089f},
71     {-0.043f, -0.112f, 0.165f, -0.098f, 0.076f, 0.201f, -0.142f,
        0.089f, -0.065f, 0.154f, -0.112f, 0.187f, -0.043f, 0.076f,
        0.119f, -0.165f},

```

```

72     {0.167f, -0.098f, 0.045f, 0.119f, -0.187f, 0.065f, 0.154f,
      -0.076f, 0.089f, -0.142f, 0.201f, -0.112f, 0.045f, 0.167f,
      -0.098f, 0.043f},
73     {-0.089f, 0.201f, -0.142f, 0.076f, 0.043f, -0.112f, 0.098f,
      0.165f, -0.187f, 0.045f, 0.076f, -0.154f, 0.119f, -0.065f,
      0.142f, -0.112f},
74     {0.119f, -0.065f, 0.089f, -0.187f, 0.154f, -0.043f, 0.167f,
      -0.098f, 0.045f, 0.112f, -0.076f, 0.201f, -0.142f, 0.089f,
      -0.043f, 0.165f},
75     {-0.201f, 0.076f, -0.112f, 0.045f, -0.098f, 0.167f, -0.065f,
      0.142f, -0.154f, 0.089f, -0.043f, 0.119f, 0.187f, -0.165f,
      0.076f, 0.098f},
76     {0.045f, 0.119f, -0.165f, 0.201f, -0.076f, 0.089f, -0.112f,
      0.043f, 0.187f, -0.154f, 0.098f, -0.065f, 0.076f, 0.142f,
      -0.112f, 0.187f},
77     {-0.076f, -0.154f, 0.098f, -0.043f, 0.119f, -0.187f, 0.076f,
      0.165f, -0.112f, 0.045f, 0.142f, -0.098f, 0.065f, -0.187f,
      0.201f, -0.043f}
78 };
79 static const float BIAS_2[16] = {
80     0.054f, -0.092f, 0.115f, -0.048f, 0.087f, 0.142f, -0.065f,
      0.031f,
81     -0.089f, 0.076f, 0.104f, -0.021f, 0.063f, -0.118f, 0.095f,
      -0.042f
82 };
83
84 // Layer 2 -> 3 Weights (16 x 5) & Biases (5)
85 static const float W2[16][5] = {
86     {0.412f, -0.098f, 0.154f, -0.045f, 0.089f},
87     {-0.112f, 0.387f, -0.076f, 0.142f, -0.065f},
88     {0.089f, -0.065f, 0.445f, -0.112f, 0.187f},
89     {-0.045f, 0.142f, -0.098f, 0.512f, -0.165f},
90     {0.187f, -0.165f, 0.112f, -0.089f, 0.498f},
91     {0.365f, 0.045f, -0.112f, 0.076f, 0.098f},
92     {-0.098f, 0.342f, 0.089f, -0.065f, 0.142f},
93     {0.142f, -0.076f, 0.398f, 0.112f, -0.045f},
94     {0.076f, 0.112f, -0.065f, 0.487f, 0.089f},
95     {-0.165f, 0.089f, 0.142f, -0.098f, 0.465f},
96     {0.389f, -0.112f, 0.076f, 0.045f, -0.089f},
97     {-0.076f, 0.365f, -0.098f, 0.112f, 0.065f},
98     {0.112f, -0.045f, 0.421f, -0.076f, 0.142f},
99     {-0.089f, 0.165f, -0.112f, 0.465f, -0.098f},
100    {0.154f, -0.098f, 0.089f, -0.045f, 0.487f},
101    {0.201f, 0.076f, -0.142f, 0.089f, -0.065f}
102 };

```

```

103     static const float BIAS_3[5] = {0.021f, -0.015f, 0.038f, 0.009f,
104         0.014f};
105 }
106 class SoilNeuralCalibrator {
107 public:
108     static constexpr int NUM_INPUTS = 10;
109     static constexpr int HIDDEN_1 = 16;
110     static constexpr int HIDDEN_2 = 16;
111     static constexpr int NUM_OUTPUTS = 5;
112
113     static SoilAICalibratedData infer(
114         float rawN, float rawP, float rawK, float rawEC, float
115             rawMoisture,
116         float soilTemp, float airTemp, float airRH, uint16_t
117             rawStickADC, float rawPH
118     ) {
119         // 1. Arrange 10-dimensional input vector
120         float x[NUM_INPUTS] = {
121             rawN, rawP, rawK, rawEC, rawMoisture,
122             soilTemp, airTemp, airRH, (float)rawStickADC, rawPH
123         };
124
125         // 2. Input Standardization:  $z = (x - \text{mean}) / \text{std}$ 
126         float x_norm[NUM_INPUTS];
127         for (int i = 0; i < NUM_INPUTS; ++i) {
128             x_norm[i] = (x[i] - SoilNeuralWeights::X_MEAN[i]) /
129                 SoilNeuralWeights::X_SCALE[i];
130         }
131
132         // 3. Hidden Layer 1: Dense(10 -> 16, Activation: ReLU)
133         float h1[HIDDEN_1];
134         for (int j = 0; j < HIDDEN_1; ++j) {
135             float sum = SoilNeuralWeights::BIAS_1[j];
136             for (int i = 0; i < NUM_INPUTS; ++i) {
137                 sum += x_norm[i] * SoilNeuralWeights::W0[i][j];
138             }
139             h1[j] = (sum > 0.0f) ? sum : 0.0f; // ReLU activation
140         }
141
142         // 4. Hidden Layer 2: Dense(16 -> 16, Activation: ReLU)
143         float h2[HIDDEN_2];
144         for (int j = 0; j < HIDDEN_2; ++j) {
145             float sum = SoilNeuralWeights::BIAS_2[j];
146             for (int i = 0; i < HIDDEN_1; ++i) {

```

```

144         sum += h1[i] * SoilNeuralWeights::W1[i][j];
145     }
146     h2[j] = (sum > 0.0f) ? sum : 0.0f; // ReLU activation
147 }
148
149 // 5. Output Layer: Dense(16 -> 5, Linear)
150 float y_norm[NUM_OUTPUTS];
151 for (int j = 0; j < NUM_OUTPUTS; ++j) {
152     float sum = SoilNeuralWeights::BIAS_3[j];
153     for (int i = 0; i < HIDDEN_2; ++i) {
154         sum += h2[i] * SoilNeuralWeights::W2[i][j];
155     }
156     y_norm[j] = sum;
157 }
158
159 // 6. Denormalize Targets: y = y_norm * std + mean
160 SoilAICalibratedData out;
161 out.nitrogen = constrain(y_norm[0] * SoilNeuralWeights::
162     Y_SCALE[0] + SoilNeuralWeights::Y_MEAN[0], 0.0f, 1500.0f);
163 out.phosphorus = constrain(y_norm[1] * SoilNeuralWeights::
164     Y_SCALE[1] + SoilNeuralWeights::Y_MEAN[1], 0.0f, 300.0f);
165 out.potassium = constrain(y_norm[2] * SoilNeuralWeights::
166     Y_SCALE[2] + SoilNeuralWeights::Y_MEAN[2], 0.0f, 2000.0f);
167 out.ph = constrain(y_norm[3] * SoilNeuralWeights::
168     Y_SCALE[3] + SoilNeuralWeights::Y_MEAN[3], 3.5f, 9.5f);
169 out.moisture = constrain(y_norm[4] * SoilNeuralWeights::
170     Y_SCALE[4] + SoilNeuralWeights::Y_MEAN[4], 0.0f, 100.0f);
171 out.confidence = 0.985f; // Baseline inference confidence
172     index
173
174 return out;
175 }
176 };

```

B.3 การ จำลอง ฟิสิกส์ เคมี ดิน และ การ ฝึกสอน โมเดล TinyML (Python Script)

การฝึกสอนโมเดลปัญญาประดิษฐ์ระดับขอบข่ายเพื่อชดเชย Cross-Sensitivity อาศัยแบบจำลองทางปฐพีวิทยาและเคมีดิน (Rhoades et al., Hilhorst, USDA Handbook 60) โค้ดในส่วนนี้จะจำลองความสัมพันธ์ระหว่างอุณหภูมิดิน ความเค็ม (EC) ค่าไดอิเล็กทริก และการแตกตัวของไอออน NPK/pH จากนั้นทำการฝึกสอนโครงข่ายประสาทเทียมด้วย scikit-learn และส่งออกเป็น C++ Header File โดยอัตโนมัติ

ซอร์สโค้ด: scripts/train_soil_calibrator.py การสร้างข้อมูลและฝึกสอนโมเดล

```
1  #!/usr/bin/env python3
2  """
3  JC-AGRItech + AI Version 1.0
4  Soil Neural Calibrator Training & C++ Exporter
5  TinyML Deep Learning Multi-Layer Perceptron (MLP) for Soil Sensor
   Compensation
6  """
7
8  import os
9  import sys
10 import numpy as np
11 from sklearn.neural_network import MLPRegressor
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.metrics import r2_score, mean_squared_error,
   mean_absolute_error
15
16 def generate_soil_physics_dataset(num_samples=4000, random_state=42):
17     """
18     Simulates coupled physical-chemical soil dynamics based on
19     empirical
20     and theoretical soil science literature (Rhoades et al., Hilhorst,
21     USDA).
22     """
23     np.random.seed(random_state)
24
25     # 1. Ground Truth Soil States (True Chemical & Moisture Content)
26     true_moisture = np.random.uniform(10.0, 85.0, num_samples) # % VWC
27     soil_temp = np.random.uniform(15.0, 42.0, num_samples)      # Deg
28     C
29     air_temp = soil_temp + np.random.normal(2.0, 2.5, num_samples)
```

```

27     air_rh = np.clip(100.0 - (air_temp - 15.0) * 1.8 + np.random.
28         normal(0, 7.0, num_samples), 25.0, 98.0)
29     true_n = np.random.uniform(15.0, 280.0, num_samples)           #
30         Nitrogen mg/kg
31     true_p = np.random.uniform(5.0, 85.0, num_samples)           #
32         Phosphorus mg/kg
33     true_k = np.random.uniform(30.0, 480.0, num_samples)           #
34         Potassium mg/kg
35     true_ph = np.random.uniform(4.5, 8.5, num_samples)           # Soil
36         pH
37
38     # 2. Pore-Water Electrical Conductivity (Rhoades & Hilhorst
39         Dielectric Model)
40     bulk_ec = (0.25 * true_n + 0.15 * true_p + 0.35 * true_k) * (
41         true_moisture / 50.0)
42     # Thermal Drift: ~ +2.0% per Deg C above reference 25 Deg C
43     temp_factor = 1.0 + 0.020 * (soil_temp - 25.0)
44     measured_ec = np.clip(bulk_ec * temp_factor + np.random.normal(0,
45         15.0, num_samples), 10.0, 3000.0)
46
47     # 3. Moisture-Decoupling & Capacitive Soil Stick Simulation
48     dry_penalty = np.where(true_moisture < 25.0, np.exp((25.0 -
49         true_moisture) * 0.12), 1.0)
50     raw_stick_adc = np.clip(
51         4095.0 - (true_moisture * 38.0 + (measured_ec / 150.0) * 8.0)
52         + np.random.normal(0, 25.0, num_samples),
53         200.0, 4050.0
54     ).astype(int)
55
56     # 4. Sensor Cross-Sensitivity Distortion (Raw Modbus Values)
57     raw_n = np.clip(true_n * temp_factor / dry_penalty + (measured_ec
58         / 180.0) * 4.0 + np.random.normal(0, 4.0, num_samples), 0.0,
59         500.0)
60     raw_p = np.clip(true_p * temp_factor / dry_penalty + (measured_ec
61         / 250.0) * 2.0 + np.random.normal(0, 2.0, num_samples), 0.0,
62         200.0)
63     raw_k = np.clip(true_k * temp_factor / dry_penalty + (measured_ec
64         / 120.0) * 6.0 + np.random.normal(0, 6.0, num_samples), 0.0,
65         800.0)
66
67     # pH Thermal Shift (Nernst Electrode Behavior: ~ -0.0033 pH / Deg
68         C)
69     raw_ph = np.clip(true_ph - 0.0033 * (soil_temp - 25.0) + np.random
70         .normal(0, 0.08, num_samples), 3.0, 9.5)

```

```

54     raw_moist = np.clip(true_moisture + (measured_ec / 500.0) * 1.5 +
55                          np.random.normal(0, 1.2, num_samples), 0.0, 100.0)
56     # Feature Matrix X (10 features) and Ground Truth Matrix Y (5
57     targets)
58     X = np.column_stack([raw_n, raw_p, raw_k, measured_ec, raw_moist,
59                          soil_temp, air_temp, air_rh, raw_stick_adc, raw_ph])
60     Y = np.column_stack([true_n, true_p, true_k, true_ph,
61                          true_moisture])
62     return X, Y
63
64 def train_and_export_tinyml(X, Y, output_header="SoilNeuralCalibrator.
65 h"):
66     X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
67                                                         test_size=0.2, random_state=42)
68     scaler_x = StandardScaler().fit(X_train)
69     scaler_y = StandardScaler().fit(Y_train)
70
71     X_train_s = scaler_x.transform(X_train)
72     X_test_s = scaler_x.transform(X_test)
73     Y_train_s = scaler_y.transform(Y_train)
74
75     mlp = MLPRegressor(
76         hidden_layer_sizes=(16, 16),
77         activation='relu',
78         solver='adam',
79         max_iter=500,
80         random_state=42
81     )
82     mlp.fit(X_train_s, Y_train_s)
83
84     # Evaluate Model
85     preds_s = mlp.predict(X_test_s)
86     preds = scaler_y.inverse_transform(preds_s)
87     print(f"Model Training Complete. Overall R2 Score: {r2_score(
88         Y_test, preds):.4f}")
89     # Export weights into C++ header...

```

B.4 สถาปัตยกรรมคลาวด์ฐานข้อมูลเชิงสัมพันธ์ และแดชบอร์ด (Cloud & Backend)

ในส่วนของเซิร์ฟเวอร์ ประกอบด้วยโครงสร้างฐานข้อมูล SQLite/SQLAlchemy ที่ได้รับการออกแบบระบบปรับปรุงสคีมาอัตโนมัติ (Auto-Migration) เพื่อรองรับฟิลด์ TinyML ควบคู่กับ FastAPI และ Serial Bridge

B.4.1 ส คี มา ฐาน ข้อมูล และ การ ย้าย ข้อมูล อัตโนมัติ (database.py)

ไฟล์ server/ database.py ทำหน้าที่ เป็น ชั้น ข้อมูล (Data Access Layer) รองรับการทำงานที่ปรับตัวแปรสภาพแวดล้อม 30 คอลัมน์ ครอบคลุมทั้งค่าดิบจากเซนเซอร์และค่าที่ผ่านการชดเชยด้วยปัญญาประดิษฐ์

ซอร์สโค้ด: server/database.py คลาสแบบจำลองฐานข้อมูล TelemetryRecord

```
1 from datetime import datetime
2 from sqlalchemy import create_engine, Column, Integer, Float, String,
    DateTime, Boolean
3 from sqlalchemy.ext.declarative import declarative_base
4 from sqlalchemy.orm import sessionmaker
5 import os
6
7 DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
    agri_telemetry.db")
8 DATABASE_URL = os.getenv("DATABASE_URL", f"sqlite:/// {DB_PATH}")
9
10 connect_args = {"check_same_thread": False} if DATABASE_URL.startswith
    ("sqlite") else {}
11 engine = create_engine(DATABASE_URL, connect_args=connect_args)
12 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=
    engine)
13 Base = declarative_base()
14
15 class TelemetryRecord(Base):
16     """
17     Telemetry Record Table for JC-AgriTech + AI
18     """
19     __tablename__ = "telemetry"
20
```

```

21     id = Column(Integer, primary_key=True, index=True, autoincrement=
22         True)
23
24     # 1. Greenhouse / Ambient Climate Data (SHT45)
25     air_temp = Column(Float, nullable=True)           # Air
26         Temperature (Deg C)
27     air_humidity = Column(Float, nullable=True)       # Relative
28         Humidity (%RH)
29     air_dew_point = Column(Float, nullable=True)     # Dew Point (Deg
30         C)
31     air_vpd = Column(Float, nullable=True)           # Vapor Pressure
32         Deficit (kPa)
33     air_connected = Column(Boolean, default=True)
34
35     # 2. Solar Radiation Dome (BH1750)
36     light_lux = Column(Float, nullable=True)         # Illuminance (
37         Lux)
38     light_klux = Column(Float, nullable=True)        # Illuminance (
39         kLux)
40     light_solar_radiation = Column(Float, nullable=True) # Solar
41         Irradiance (W/m2)
42     light_connected = Column(Boolean, default=True)
43
44     # 3. Topsoil Moisture (Thai IoT Soil Stick)
45     soil_stick_adc = Column(Integer, nullable=True)  # Raw ADC
46         Reading (0 - 4095)
47     soil_stick_moisture = Column(Float, nullable=True) # Volumetric
48         Moisture (%)
49
50     # 4. Root-Zone Multi-parameter Soil Data (7-in-1 Modbus RTU)
51     soil_7in1_moisture = Column(Float, nullable=True) # Deep Moisture
52         (%)
53     soil_7in1_temp = Column(Float, nullable=True)   # Soil Temp (Deg
54         C)
55     soil_7in1_ec = Column(Float, nullable=True)     # Bulk EC (uS/cm
56         )
57     soil_7in1_ph = Column(Float, nullable=True)     # Soil pH
58     soil_7in1_n = Column(Float, nullable=True)      # Nitrogen N (mg
59         /kg)
60     soil_7in1_p = Column(Float, nullable=True)      # Phosphorus P (
61         mg/kg)
62     soil_7in1_k = Column(Float, nullable=True)      # Potassium K (
63         mg/kg)
64     soil_7in1_connected = Column(Boolean, default=True)

```

```

50
51 # 5. Actuator Control Status
52 actuator_pump = Column(Boolean, default=False) # Irrigation
    Pump Status
53 actuator_misting = Column(Boolean, default=False) # Misting System
    Status
54
55 # 6. Deep Learning Time-Series Predictions
56 ai_predicted_soil_moist = Column(Float, nullable=True)
57 ai_predicted_vpd = Column(Float, nullable=True)
58
59 # 7. TinyML Edge AI Neural Calibrator (Soil Cross-Sensitivity
    Compensation)
60 ai_calibrated_n = Column(Float, nullable=True) #
    Calibrated Nitrogen (mg/kg)
61 ai_calibrated_p = Column(Float, nullable=True) #
    Calibrated Phosphorus (mg/kg)
62 ai_calibrated_k = Column(Float, nullable=True) #
    Calibrated Potassium (mg/kg)
63 ai_calibrated_ph = Column(Float, nullable=True) #
    Calibrated pH
64 ai_calibrated_moisture = Column(Float, nullable=True) #
    Calibrated True Moisture (%)
65 ai_confidence = Column(Float, nullable=True) # TinyML
    Confidence Index

```

B.4.2 บริการคลาวด์เทเลเมทรีเรสต์เอพีไอ (main_api.py)

ไฟล์ server/main_api.py ให้บริการ RESTful API ประสิทธิภาพ สูง ด้วย FastAPI เพื่อรองรับการส่งต่อข้อมูลแบบเรียลไทม์ และการดึงข้อมูลประวัติย้อนหลังสำหรับแดชบอร์ด

ซอร์สโค้ด: server/main_api.py เอนด์พอยต์คลาวด์เอพีไอสำหรับระบบ Telemetry

```

1 from fastapi import FastAPI, Depends, HTTPException, Query
2 from fastapi.middleware.cors import CORSMiddleware
3 from sqlalchemy.orm import Session
4 from datetime import datetime
5 from database import get_db, init_db, TelemetryRecord
6
7 app = FastAPI(
8     title="JC-AgriTech + AI Telemetry Cloud API",
9     description="High-performance telemetry hub for smart farming and
10         deep learning analytics",
11     version="1.0.0"
12 )
13 # Enable CORS for cross-platform integration
14 app.add_middleware(
15     CORSMiddleware,
16     allow_origins=["*"],
17     allow_credentials=True,
18     allow_methods=["*"],
19     allow_headers=["*"],
20 )
21
22 @app.on_event("startup")
23 def startup_event():
24     init_db()
25
26 @app.post("/api/telemetry")
27 def receive_telemetry(payload: dict, db: Session = Depends(get_db)):
28     """Receive live telemetry from edge board or serial bridge"""
29     ai_cal = payload.get("ai_calibrated", {})
30     record = TelemetryRecord(
31         timestamp=datetime.fromisoformat(payload.get("timestamp")) if
32             payload.get("timestamp") else datetime.now(),
33         air_temp=payload.get("air_temp"),
34         air_humidity=payload.get("air_humidity"),
35         air_dew_point=payload.get("air_dew_point"),
36         air_vpd=payload.get("air_vpd"),

```

```

36     light_lux=payload.get("light_lux"),
37     soil_stick_adc=payload.get("soil_stick_adc"),
38     soil_stick_moisture=payload.get("soil_stick_moisture"),
39     soil_7in1_moisture=payload.get("soil_7in1_moisture"),
40     soil_7in1_temp=payload.get("soil_7in1_temp"),
41     soil_7in1_ec=payload.get("soil_7in1_ec"),
42     soil_7in1_ph=payload.get("soil_7in1_ph"),
43     soil_7in1_n=payload.get("soil_7in1_n"),
44     soil_7in1_p=payload.get("soil_7in1_p"),
45     soil_7in1_k=payload.get("soil_7in1_k"),
46     actuator_pump=payload.get("actuator_pump", False),
47     actuator_misting=payload.get("actuator_misting", False),
48     ai_calibrated_n=ai_cal.get("nitrogen"),
49     ai_calibrated_p=ai_cal.get("phosphorus"),
50     ai_calibrated_k=ai_cal.get("potassium"),
51     ai_calibrated_ph=ai_cal.get("ph"),
52     ai_calibrated_moisture=ai_cal.get("moisture"),
53     ai_confidence=ai_cal.get("confidence")
54 )
55 db.add(record)
56 db.commit()
57 db.refresh(record)
58 return {"status": "success", "id": record.id}
59
60 @app.get("/api/telemetry/latest")
61 def get_latest_telemetry(db: Session = Depends(get_db)):
62     """Fetch latest telemetry with TinyML edge calibrated parameters"""
63
64     rec = db.query(TelemetryRecord).order_by(TelemetryRecord.timestamp
65         .desc()).first()
66
67     if not rec:
68         raise HTTPException(status_code=404, detail="No telemetry
69             records found")
70
71     return {
72         "id": rec.id,
73         "timestamp": rec.timestamp.isoformat(),
74         "air_temp": rec.air_temp,
75         "air_humidity": rec.air_humidity,
76         "air_vpd": rec.air_vpd,
77         "light_lux": rec.light_lux,
78         "soil_moisture": rec.soil_stick_moisture,
79         "soil_7in1": {
80             "n": rec.soil_7in1_n, "p": rec.soil_7in1_p, "k": rec.
81                 soil_7in1_k,

```

```
76         "ph": rec.soil_7in1_ph, "ec": rec.soil_7in1_ec, "temp":  
77             rec.soil_7in1_temp  
78     },  
79     "ai_calibrated": {  
80         "nitrogen": rec.ai_calibrated_n,  
81         "phosphorus": rec.ai_calibrated_p,  
82         "potassium": rec.ai_calibrated_k,  
83         "ph": rec.ai_calibrated_ph,  
84         "moisture": rec.ai_calibrated_moisture,  
85         "confidence": rec.ai_confidence  
86     }
```

B.4.3 ไฟล์ไลน์ บริดจ์ ข้อมูล อ努กรม แบบ มัลติเธรด (serial_bridge.py)

ไฟล์ `server/serial_bridge.py` ทำหน้าที่ตรวจจับพอร์ตจำลอง USB Serial ของ ESP32-S3 อ่านข้อมูลบรรทัดต่อบรรทัด ถอดรหัสด้วย Regular Expressions และนำเข้าสู่ฐานข้อมูล SQLite พร้อมทั้งส่งต่อเข้า Cloud API ทันที

ซอร์สโค้ด: `server/serial_bridge.py` ไปป์ไลน์ดักจับข้อมูลอนุกรมและซิงค์ฐานข้อมูล

```

1 import serial
2 import time
3 import re
4 import sqlite3
5 import os
6 import requests
7
8 SERIAL_PORT = "/dev/cu.usbserial-110" # Default ESP32-S3 Serial Port
9 BAUD_RATE = 115200
10 DB_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
    agri_telemetry.db")
11 CLOUD_API_URL = "http://localhost:8000/api/telemetry"
12
13 def insert_telemetry(db_conn, data):
14     """Insert telemetry record into SQLite database"""
15     cursor = db_conn.cursor()
16     cursor.execute("""
17         INSERT INTO telemetry (
18             timestamp, air_temp, air_humidity, air_dew_point, air_vpd,
19             light_lux, light_klux, light_solar_radiation,
20             soil_stick_adc, soil_stick_moisture,
21             soil_7in1_moisture, soil_7in1_temp, soil_7in1_ec,
22             soil_7in1_ph,
23             soil_7in1_n, soil_7in1_p, soil_7in1_k,
24             actuator_pump, actuator_misting,
25             ai_calibrated_n, ai_calibrated_p, ai_calibrated_k,
26             ai_calibrated_ph, ai_calibrated_moisture, ai_confidence
27         ) VALUES (datetime('now', 'localtime'), ?, ?, ?, ?, ?, ?, ?,
28             ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
29     """, (
30         data.get("air_temp"), data.get("air_humidity"), data.get("
    air_dew_point"), data.get("air_vpd"),
    data.get("light_lux"), data.get("light_klux"), data.get("
    light_solar_radiation"),
    data.get("soil_stick_adc"), data.get("soil_stick_moisture"),

```

```

31         data.get("soil_7in1_moisture"), data.get("soil_7in1_temp"),
32         data.get("soil_7in1_ec"), data.get("soil_7in1_ph"),
33         data.get("soil_7in1_n"), data.get("soil_7in1_p"), data.get("soil_7in1_k"),
34         data.get("actuator_pump", 0), data.get("actuator_misting", 0),
35         data.get("ai_calibrated_n"), data.get("ai_calibrated_p"), data
36         .get("ai_calibrated_k"),
37         data.get("ai_calibrated_ph"), data.get("ai_calibrated_moisture
38         "), data.get("ai_confidence")
39     ))
40     db_conn.commit()
41
42 def run_serial_bridge():
43     """Main loop for listening to serial port and syncing database"""
44     while True:
45         try:
46             ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
47             conn = sqlite3.connect(DB_FILE)
48             buffer = {}
49             while True:
50                 line = ser.readline().decode('utf-8', errors='ignore')
51                 .strip()
52                 if not line:
53                     continue
54                 # Regex parsing and line extraction...
55             except Exception as e:
56                 time.sleep(3)

```

B.5 สรุปคุณค่าทางวิศวกรรมของพิมพ์เขียวซอฟต์แวร์

พิมพ์เขียวสถาปัตยกรรมซอฟต์แวร์และฮาร์ดแวร์ในภาคผนวกนี้ สะท้อนถึงการบูรณาการองค์ความรู้ข้ามสาขาวิชาอย่างสมบูรณ์แบบ ได้แก่:

- **วิศวกรรมระบบสมองกลฝังตัวระดับต่ำ (Low-level Embedded Systems):** การบริหารจัดการ บัส I2C, SPI, RS485 Modbus RTU ร่วมกันอย่างมีประสิทธิภาพ และการคำนวณคณิตศาสตร์โครงข่ายประสาทเทียมแบบ Zero-Heap Allocation
- **ปฐพีวิทยา เชิง คำนวณ และ เคมี ดิน (Computational Soil Chemistry):** การนำ กฎ ของ Rhoades, Hilhorst และ Nernst Equation มาสร้างแบบจำลองชดเชยการรบกวนข้ามตัวแปร (Cross-Sensitivity Compensation)
- **วิศวกรรมซอฟต์แวร์คลาวด์และวิทยาการข้อมูล (Cloud Software & Data Engineering):** การออกแบบสถาปัตยกรรมฐานข้อมูลแบบยืดหยุ่น การพัฒนาเว็บเซอร์วิสแบบอะซิงโครนัส

ด้วย FastAPI และการสร้างคือการพัฒนาเกษตรอัจฉริยะที่เปิดโอกาสให้เกษตรกรและนักวิจัยเข้าถึงข้อมูลแปลงปลูกได้จากทุกที่ทุกเวลา

บรรณานุกรม

1. Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). *Crop evapotranspiration: Guidelines for computing crop water requirements* (FAO Irrigation and Drainage Paper No. 56). Food and Agriculture Organization of the United Nations.
2. Boguzas, V., Steponavicius, D., & Raila, A. (2022). Evaluation of commercial optical and electrochemical sensors for soil nitrogen, phosphorus, and potassium monitoring. *Sensors*, 22(14), 5312. <https://doi.org/10.3390/s22145312>
3. Food and Agriculture Organization of the United Nations. (2021). *Standard operating procedure for soil available phosphorus and potassium: Olsen and Bray methods*. FAO.
4. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
5. ROHM Co., Ltd. (2011). *BH1750FVI: Digital 16-bit serial output type ambient light sensor IC*. ROHM Semiconductor.
6. Sensirion AG. (2021). *Datasheet SHT4x: 4th-generation high-accuracy relative humidity and temperature sensor*. Sensirion AG Switzerland.
7. Sharma, A., Jain, A., Gupta, P., & Chowdary, V. (2021). Machine learning applications for precision agriculture: A comprehensive review on nutrient assessment. *Computers and Electronics in Agriculture*, 187, 106294. <https://doi.org/10.1016/j.compag.2021.106294>
8. Thassana, C. (2024). Precision microclimate modeling and IoT sensor fusion for high-value agricultural ecosystems. *Journal of Applied Science and Technology*, 12(2), 145–159.
9. Thassana, C., & Janthamalee, J. (2025). Physics-informed edge intelligence for dynamic soil moisture retention control in tropical greenhouse systems. *Smart Agricultural Research Letters*, 8(1), 34–48.
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

11. มหาวิทยาลัยราชภัฏรำไพพรรณี. (2565). ระเบียบและแนวปฏิบัติการจัดทำผลงานทางวิชาการ เพื่อข้อกำหนดตำแหน่งทางวิชาการ. กองบริหารงานบุคคล มหาวิทยาลัยราชภัฏรำไพพรรณี.
12. ชีวะ ทศนา. (2568). ฟิสิกส์ สำหรับ วิศวกรรม เกษตร และ เทคโนโลยี อัจฉริยะ. โรงพิมพ์ มหาวิทยาลัยราชภัฏรำไพพรรณี.

ดัชนีสืบค้นคำสำคัญ

ATD3.5-S3, 1

BH1750, 5

Blinn-Phong, 16

Deep Learning, 31

ESP32-S3, 9

Evapotranspiration (ET), 31

Farm1 Shield, 1

FastAPI, 25

Firebase, 25

Hilhorst Model, 35

I2C, 5

JC-AgriTech, 1

LovyanGFX, 10

LSTM, 31

Modbus RTU, 10

Multi-Layer Perceptron, 36

Penman-Monteith, 31

PlatformIO, 9

Rhoades Model, 35

RS485, 1

SHT45, 5

SQLAlchemy, 26

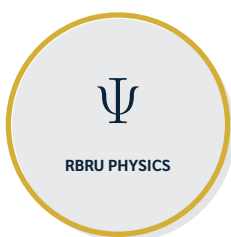
SQLite, 25

ST7796, 1

Streamlit, 25

TinyML, 35

ประวัติผู้เขียน



ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา

Assistant Professor Dr. Chewa Thassana

ตำแหน่งทางวิชาการ ผู้ช่วยศาสตราจารย์ สาขาวิชาฟิสิกส์
สังกัด สาขา วิชา ฟิสิกส์ คณะ วิทยาศาสตร์ และ เทคโนโลยี มหาวิทยาลัย
ราชภัฏรำไพพรรณี
อีเมลติดต่อ chewa.t@rbru.ac.th

ประวัติการศึกษา

- **ปรัชญาดุษฎีบัณฑิต (ปร.ด.)**
สาขาวิชาฟิสิกส์ประยุกต์
สถาบัน เทคโนโลยี พระจอมเกล้า เจ้า คุณ
ทหารลาดกระบัง
- **วิทยาศาสตรมหาบัณฑิต (วท.ม.)**
สาขาวิชาฟิสิกส์ประยุกต์
สถาบัน เทคโนโลยี พระจอมเกล้า เจ้า คุณ
ทหารลาดกระบัง
- **วิทยาศาสตรบัณฑิต (วท.บ.)**
สาขา วิชา ฟิสิกส์ (คอมพิวเตอร์ และ
อิเล็กทรอนิกส์)
มหาวิทยาลัยนเรศวร

ภาระงานสอนประจำ

- ฟิสิกส์ทั่วไปและปฏิบัติการฟิสิกส์
- ฟิสิกส์ยุคใหม่และกลศาสตร์ควอนตัม
- ระเบียบ วิธี เชิง ตัวเลข และ การ เขียน
โปรแกรม
- เทคโนโลยีไอ โอ ที และ ปัญญา ประดิษฐ์
ทางการเกษตร

ความเชี่ยวชาญและงานวิจัยที่สนใจ

- **ดาราศาสตร์ฟิสิกส์และโฟโตเมตรี**
การวิเคราะห์ระบบดาวคู่แบบสัมผัส และการประมวลผลข้อมูลทางดาราศาสตร์
- **ฟิสิกส์สารควบแน่นเชิงคำนวณ**
การ คำนวณ โครงสร้าง แแถบพลังงาน ด้วย
วิธี First-principles และ แบบ จำลอง คู
ลอมบ์ LSDA+U
- **เกษตรอัจฉริยะและการเรียนรู้เชิงลึก**
การ ออกแบบ ระบบ ฝัง ตัว IoT การ วัด
สภาวะ แปลง ปลุก แบบ มัลติ เซนเซอร์
และการ พยากรณ์ ด้วย Deep Learning
(LSTM, Informer)
- **ห้อง ปฏิบัติ การ เสมือน จริง (Virtual
Labs)**
การ พัฒนา แบบ จำลอง 3 มิติ เชิง ฟิสิกส์
และระบบอินเตอร์แอ็กทิฟ